

Expression model - notebook export

This PDF contains the notebook's stored notes, code and outputs. No cells were rerun. Missing outputs are not recreated.

Task 2 - Expression model - Google Colab

Course: DLBAIPEAI - Project Edge AI

Student: W. Pretorius

Platform: Google Colab

This notebook is the Google Colab version of the expression model for **Task 2**. It trains an EfficientNetV2-B0 model to recognise seven facial expressions.

The classes are Angry, Disgust, Fear, Happy, Sad, Surprise and Neutral.

This version keeps the same group-aware data split, recall checks, balanced-accuracy model selection and TensorFlow Lite checks as the desktop notebook.

Before running the notebook, choose **Runtime -> Change runtime type -> GPU** in Google Colab.

Code cell 2

```
# cell:folder
import os
import sys
from pathlib import Path
from datetime import datetime

PROJECT_DIR = Path("/content/project_edge_ai")
PROJECT_DIR.mkdir(
    parents=True,
    exist_ok=True,
)

TASK = "expression"
NOTEBOOK_NAME = "03_Expression_Model_Colab"

OUTPUT_DIR = (
    PROJECT_DIR
    / "results"
    / TASK
    / datetime.now().strftime("%Y%m%d_%H%M%S")
)

OUTPUT_DIR.mkdir(
    parents=True,
    exist_ok=True,
)

os.environ["KERAS_HOME"] = str(
    PROJECT_DIR
    / "datasets"

    / "keras_cache"
)

os.environ["KAGGLEHUB_CACHE"] = str(
    PROJECT_DIR
    / "datasets"
    / "kagglehub_cache"
)

os.environ["TF_CPP_MIN_LOG_LEVEL"] = "2"

print("Python:", sys.version.split()[0])
print("Project folder:", PROJECT_DIR)
print("Results folder:", OUTPUT_DIR)

Python: 3.13.15
```

Project folder: /content/project_edge_ai
Results folder: /content/project_edge_ai/results/expression/20260915_092434

2. Install the extra libraries

In this cell we install the small helper libraries used by the notebook. TensorFlow itself comes from the Colab runtime.

Code cell 4

```
# cell:install
import subprocess
import sys
from importlib import metadata

# Colab already supplies TensorFlow and GPU support.
# Only small helper packages are installed here.
packages = {
    "kagglehub": "0.3.13",
    "tqdm": "4.67.1",
    "reportlab": "4.3.1",
    "nbformat": "5.10.4",
}

missing = []

for package, version in packages.items():
    try:
        installed = metadata.version(
            package
        )
    except metadata.PackageNotFoundError:
        installed = None

    if installed != version:
        missing.append(
            f"{package}=={version}"
        )

if missing:
    subprocess.check_call(
        [
            sys.executable,
            "-m",
            "pip",
            "install",
            "--quiet",
            "--disable-pip-version-check",
            *missing,
        ]
    )

print("Colab helper libraries are ready.")

Colab helper libraries are ready.
```

3. Keep the outputs for the PDF

In this cell we start recording later cell outputs for the final PDF. The code is kept here, not in a separate file. Setup-cell text is included, but its earlier output is not recorded.

Code cell 6

```
# cell:record
# This small tool stays inside the notebook. No helper file is needed.
import io
import re
import sys
import copy
import json
```

```

if "pdf_recorder" in globals():
    pdf_recorder.close()

class LiveNotebook:
    def __init__(self):
        self.ip = get_ipython()
        self.records = {}
        self.current = None
        self.stdout, self.stderr = sys.stdout, sys.stderr
        self.publish = self.ip.display_pub.publish
        self.format_data = self.ip.displayhook.write_format_data
        self.ip.display_pub.publish = self.capture_display
        self.ip.displayhook.write_format_data = self.capture_result
        self.ip.events.register("pre_run_cell", self.before)
        self.ip.events.register("post_run_cell", self.after)

    def before(self, info):
        self.restore()
        key = info.raw_cell.splitlines()[0].strip()
        self.current = key if key.startswith("# cell:") else None
        if self.current:
            self.records[self.current] = {
                "source": info.raw_cell,
                "execution_count": self.ip.execution_count,
                "outputs": [],
            }
        recorder = self
        class Tee:
            def __init__(self, original, name):
                self.original, self.name = original, name
            def write(self, text):
                outputs = recorder.records[recorder.current]["outputs"]
                if outputs and outputs[-1].get("name") == self.name:
                    outputs[-1]["text"] += text
                else:
                    outputs.append({"output_type": "stream", "name": self.name,
                                   "text": text})
                return self.original.write(text)
            def flush(self):
                return self.original.flush()
            def __getattr__(self, name):
                return getattr(self.original, name)
        sys.stdout, sys.stderr = Tee(self.stdout, "stdout"), Tee(self.stderr, "stderr")

    def capture_display(self, data, metadata=None, **kwargs):
        if self.current:
            self.records[self.current]["outputs"].append({
                "output_type": "display_data", "data": copy.deepcopy(data),
                "metadata": metadata or {},
            })
        return self.publish(data=data, metadata=metadata, **kwargs)

    def capture_result(self, data, metadata=None):
        if self.current:
            self.records[self.current]["outputs"].append({
                "output_type": "execute_result", "data": copy.deepcopy(data),
                "metadata": metadata or {}, "execution_count": self.ip.execution_count,
            })
        return self.format_data(data, metadata)

    def restore(self):
        sys.stdout, sys.stderr = self.stdout, self.stderr

    def after(self, result):
        if self.current:
            error = result.error_before_exec or result.error_in_exec
            if error:
                self.records[self.current]["outputs"].append({
                    "output_type": "stream", "name": "stderr",
                    "text": f"{type(error).__name__}: {error}",
                })
        self.restore()
        self.current = None

```

```

def close(self):
    self.restore()
    self.ip.display_pub.publish = self.publish
    self.ip.displayhook.write_format_data = self.format_data
    for event, function in [("pre_run_cell", self.before), ("post_run_cell", self.after)]:
        if function in self.ip.events.callbacks[event]:
            self.ip.events.unregister(event, function)

pdf_recorder = LiveNotebook()
print("Later cell outputs will be included in the PDF.")

Later cell outputs will be included in the PDF.

```

4. Check that the Colab GPU can train

In this cell we check that Colab can see a GPU and run a real training calculation. We also enable mixed precision to make EfficientNetV2 training faster.

Code cell 8

```

# cell:imports
import time
import hashlib
import zipfile
import shutil
import platform
from concurrent.futures import ThreadPoolExecutor

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from PIL import Image
from tqdm import tqdm
import kagglehub
import tensorflow as tf

from sklearn.model_selection import StratifiedGroupKFold
from sklearn.metrics import (
    accuracy_score,
    balanced_accuracy_score,
    f1_score,
    classification_report,
    confusion_matrix,
    ConfusionMatrixDisplay,
)

from IPython.display import display, Markdown

get_ipython().run_line_magic(
    "matplotlib",
    "inline",
)

pd.set_option(
    "display.max_columns",
    12,
)

pd.set_option(
    "display.width",
    120,
)

print("TensorFlow:", tf.__version__)
print("System:", platform.platform())

gpus = tf.config.list_physical_devices(
    "GPU"
)

if not gpus:
    raise RuntimeError(
        "No GPU was found. In Colab choose Runtime -> Change runtime type -> GPU, "

```

```

        "then run the notebook again."
    )

for gpu in gpus:
    try:
        tf.config.experimental.set_memory_growth(
            gpu,
            True,
        )
    except RuntimeError:
        pass

GPU_NAME = tf.config.experimental.get_device_details(
    gpus[0]
).get(
    "device_name",
    str(gpus[0]),
)

print("GPU found:", GPU_NAME)

# Check a real forward and backward calculation.
previous_placement = tf.config.get_soft_device_placement()
tf.config.set_soft_device_placement(
    False
)

try:
    with tf.device("/GPU:0"):
        probe_images = tf.random.normal(
            (2, 32, 32, 3)
        )

        probe_weights = tf.Variable(
            tf.random.normal(
                (3, 3, 3, 8)
            )
        )

        with tf.GradientTape() as tape:
            probe_output = tf.nn.conv2d(
                probe_images,
                probe_weights,
                strides=1,
                padding="SAME",
            )

            probe_loss = tf.reduce_mean(
                tf.square(
                    probe_output
                )
            )

            probe_gradient = tape.gradient(
                probe_loss,
                probe_weights,
            )

        if probe_gradient is None:
            raise RuntimeError(
                "The GPU did not calculate a gradient."
            )

        if "GPU" not in probe_output.device.upper():
            raise RuntimeError(
                "The test calculation did not run on the GPU."
            )

        if not np.isfinite(
            probe_gradient.numpy()
        ).all():
            raise RuntimeError(
                "The GPU gradient contains invalid values."
            )

```

```

    )

finally:
    tf.config.set_soft_device_placement(
        previous_placement
    )

GPU_READY = True

# EfficientNetV2 is much faster with mixed precision on Colab GPUs.
tf.keras.mixed_precision.set_global_policy(
    "mixed_float16"
)

print(
    "Precision policy:",
    tf.keras.mixed_precision.global_policy().name,
)

print(
    "GPU forward and backward test: PASSED"
)

TensorFlow: 2.20.0
System: Linux-6.6.122+-x86_64-with-glibc2.39
GPU found: Tesla T4
Precision policy: mixed_float16
GPU forward and backward test: PASSED

```

Expression in-the-Wild dataset

For this model we use **ExpW**, or Expression in-the-Wild. It contains about 91,793 manually labelled faces with seven expression classes.

The dataset is downloaded from Kaggle inside this notebook. The original ExpW research was published by Zhang et al. The Kaggle mirror used here lists an MIT licence.

The seven labels are angry, disgust, fear, happy, sad, surprise and neutral.

5. Find or download the ExpW dataset

In this cell we check the Colab Kaggle cache first. If the ExpW labels and images are already present in this runtime, we reuse them. Kaggle is only used when a dataset is missing.

Code cell 11

```

# cell:dataset
LABEL_HANDLE = "nguhaduong/expression-in-the-wild-expw-dataset"
IMAGE_HANDLE = "minhtmnguytrn/origin-expw"

KAGGLE_CACHE = Path(
    os.environ["KAGGLEHUB_CACHE"]
)
KAGGLE_CACHE.mkdir(parents=True, exist_ok=True)

def cached_dataset(owner, slug, check_function):
    """Return the newest usable cached version, or None."""
    versions_dir = KAGGLE_CACHE / "datasets" / owner / slug / "versions"

    if not versions_dir.is_dir():
        return None

    versions = sorted(
        [p for p in versions_dir.iterdir() if p.is_dir()],
        key=lambda p: p.stat().st_mtime,
        reverse=True,
    )

    for version in versions:
        try:
            if check_function(version):

```

```

        return version.resolve()
    except Exception:
        pass

    return None

def has_expw_labels(root):
    return any(root.rglob("label.lst"))

def find_image_root(root):
    # The image Kaggle dataset normally contains an "origin" folder.
    origin_folders = [p for p in root.rglob("origin") if p.is_dir()]

    search_roots = origin_folders if origin_folders else [root]

    for candidate in search_roots:
        try:
            first_image = next(
                p for p in candidate.rglob("*")
                if p.is_file()
                and p.suffix.lower() in {".jpg", ".jpeg", ".png"}
            )
            return first_image.parent if first_image.parent == candidate else candidate
        except StopIteration:
            continue

    return None

def has_expw_images(root):
    return find_image_root(root) is not None

# -----
# Labels
# -----
LABEL_DATA_DIR = cached_dataset(
    "nguhaduong",
    "expression-in-the-wild-expw-dataset",
    has_expw_labels,
)

if LABEL_DATA_DIR is None:
    print("ExpW label dataset was not found in the cache.")
    print("Downloading the labels from Kaggle...")
    LABEL_DATA_DIR = Path(
        kagglehub.dataset_download(LABEL_HANDLE)
    ).resolve()

    if not has_expw_labels(LABEL_DATA_DIR):
        raise FileNotFoundError(
            "The label dataset downloaded, but label.lst was not found."
        )
else:
    print("Verified cached ExpW labels.")

# -----
# Original images
# -----
IMAGE_DATA_DIR = cached_dataset(
    "minhtmnguyntn",
    "origin-expw",
    has_expw_images,
)

if IMAGE_DATA_DIR is None:
    print("\nThe original ExpW images were not found in the cache.")
    print("Downloading the image archive from Kaggle.")
    print("This is a large download, about 8.4 GB on the first run.")

    IMAGE_DATA_DIR = Path(
        kagglehub.dataset_download(IMAGE_HANDLE)
    ).resolve()

```

```

    if not has_expw_images(IMAGE_DATA_DIR):
        raise FileNotFoundError(
            "The image dataset downloaded, but no JPG/PNG images were found."
        )
    else:
        print("Verified cached ExpW original images.")

IMAGE_ROOT = find_image_root(IMAGE_DATA_DIR)

if IMAGE_ROOT is None:
    raise FileNotFoundError("Could not find the ExpW image folder.")

LABEL_FILE = max(
    LABEL_DATA_DIR.rglob("label.lst"),
    key=lambda p: p.stat().st_size,
)

print("\nLabel file:")
print(LABEL_FILE)

print("\nImage folder:")
print(IMAGE_ROOT)

# Count the image files once so we can verify the dataset.
image_count = sum(
    1
    for p in IMAGE_ROOT.rglob("")
    if p.is_file()
    and p.suffix.lower() in {".jpg", ".jpeg", ".png"}
)

print("\nOriginal image files found:", image_count)

if image_count < 90000:
    raise RuntimeError(
        "The ExpW image archive looks incomplete. "
        f"Only {image_count} images were found."
    )

print("\nDataset check passed.")

ExpW label dataset was not found in the cache.
Downloading the labels from Kaggle...
Warning: Looks like you're using an outdated `kagglehub` version (installed: 0.3.13), please
consider upgrading to the latest version (1.0.2).
Downloading from https://www.kaggle.com/api/v1/datasets/download/nguhaduong/expression-in-the-wild-e
xpw-dataset?dataset_version_number=1...

100%|#####| 7.56G/7.56G [01:57<00:00, 68.9MB/s]

Extracting files...

The original ExpW images were not found in the cache.
Downloading the image archive from Kaggle.
This is a large download, about 8.4 GB on the first run.
Warning: Looks like you're using an outdated `kagglehub` version (installed: 0.3.13), please
consider upgrading to the latest version (1.0.2).
Downloading from https://www.kaggle.com/api/v1/datasets/download/minhtmguyntn/origin-expw?dataset_
version_number=1...

100%|#####| 7.68G/7.68G [01:47<00:00, 76.5MB/s]

Extracting files...

Label file:
/content/project_edge_ai/datasets/kagglehub_cache/datasets/nguhaduong/expression-in-the-wild-expw-da
taset/versions/1/expw_dataset/data/label/label.lst

Image folder:
/content/project_edge_ai/datasets/kagglehub_cache/datasets/minhtmguyntn/origin-expw/versions/1/ori
gin

Original image files found: 106962

```


Dataset check passed.

6. Read the expression labels

In this cell we connect each ExpW label to its original image. We also keep the face box so the next steps can crop the correct face from the photo.

Code cell 13

```
# cell:labels
EXPW_NAMES = {
    0: "Angry",
    1: "Disgust",
    2: "Fear",
    3: "Happy",
    4: "Sad",
    5: "Surprise",
    6: "Neutral",
}

# Build a file-name index for the original ExpW images.
print("Indexing ExpW image files...")

image_paths = [
    p.resolve()
    for p in IMAGE_ROOT.rglob("")
    if p.is_file()
    and p.suffix.lower() in {".jpg", ".jpeg", ".png"}
]

by_name = {}

for path in tqdm(image_paths, desc="Indexing images"):
    by_name.setdefault(path.name, []).append(path)

print("Unique image names indexed:", len(by_name))

rows = []
missing = []

ambiguous = []
bad_lines = 0

with LABEL_FILE.open(
    "r",
    encoding="utf-8-sig",
    errors="replace",
) as stream:

    for line in tqdm(
        stream,
        total=91793,
        desc="Reading ExpW labels",
    ):
        parts = line.strip().split()

        if len(parts) < 8:
            bad_lines += 1
            continue

        image_name = parts[0]

        try:
            face_id = int(float(parts[1]))
            top = float(parts[2])
            left = float(parts[3])
            right = float(parts[4])
            bottom = float(parts[5])
            confidence = float(parts[6])
            label = int(float(parts[7]))

        except ValueError:
            bad_lines += 1
            continue
```

```

    if label not in EXPW_NAMES:
        continue

    matches = by_name.get(image_name, [])

    if len(matches) == 0:
        missing.append(image_name)
        continue

    if len(matches) > 1:
        ambiguous.append(image_name)

    image_path = matches[0]

    width = right - left
    height = bottom - top

    rows.append({
        "path": str(image_path),
        "expression": label,
        "face_id": face_id,
        "face_x": left,
        "face_y": top,
        "face_width": width,
        "face_height": height,

        "face_confidence": confidence,
    })

raw = pd.DataFrame(rows)

print("\nLabel file:", LABEL_FILE)
print("Labelled faces loaded:", len(raw))
print("Missing image rows:", len(missing))
print("Ambiguous image names:", len(ambiguous))
print("Bad label lines:", bad_lines)

# Save a small diagnostic file only when something is missing.
OUTPUT_DIR.mkdir(parents=True, exist_ok=True)

if missing:
    pd.DataFrame(
        {"missing_image_name": sorted(set(missing))}
    ).to_csv(
        OUTPUT_DIR / "missing_expw_images.csv",
        index=False,
    )

if ambiguous:
    pd.DataFrame(
        {"ambiguous_image_name": sorted(set(ambiguous))}
    ).to_csv(
        OUTPUT_DIR / "ambiguous_expw_images.csv",
        index=False,
    )

if raw.empty:
    raise ValueError(
        "No labelled ExpW images were matched. "
        "Check that both Kaggle datasets were downloaded."
    )

# Nearly all 91,793 labelled faces should be available.
match_rate = len(raw) / 91793

print("Matched label rate:", f"{100 * match_rate:.2f}%")

if match_rate < 0.95:
    raise RuntimeError(
        "Too many ExpW labels could not be matched to images. "
        "The image archive may be incomplete or from a different dataset version."
    )

raw["expression"] = pd.to_numeric(

```

```

        raw["expression"],
        errors="raise",
    ).astype(int)

if not raw["expression"].between(0, 6).all():
    raise ValueError(
        "ExpW expression labels must be between 0 and 6."
    )

print("\nExpression counts:")

counts = (
    raw["expression"]
    .value_counts()
    .sort_index()
)

display(
    pd.DataFrame({
        "label_id": counts.index,
        "expression": [
            EXPW_NAMES[i]
            for i in counts.index
        ],
        "images": counts.values,
    })
)

print("\nTotal labelled faces:", len(raw))

Indexing ExpW image files...

Indexing images: 100%|#####| 106962/106962 [00:00<00:00, 126579.17it/s]

Unique image names indexed: 106962

Reading ExpW labels: 100%|#####| 91793/91793 [00:00<00:00, 170864.03it/s]

Label file: /content/project_edge_ai/datasets/kagglehub_cache/datasets/nguhaduong/expression-in-the-
wild-expw-dataset/versions/1/expw_dataset/data/label/label.lst
Labelled faces loaded: 91793
Missing image rows: 0
Ambiguous image names: 0
Bad label lines: 0
Matched label rate: 100.00%

Expression counts:

   label_id expression  images
0          0    Angry     3671
1          1  Disgust     3995
2          2    Fear     1088
3          3   Happy    30537
4          4     Sad    10559
5          5  Surprise     7060
6          6  Neutral    34883

Total labelled faces: 91793

```

7. Choose the training settings

In this cell we choose the image size, batch size and training length. These are starting choices. Validation data can guide changes; final holdout data must not.

Code cell 15

```

# cell:settings
SEED = 42
IMAGE_SIZE = 224
BATCH_SIZE = 128
HEAD_EPOCHS = 8
FINE_TUNE_EPOCHS = 20

CLASS_NAMES = [
    "Angry",

```

```

    "Disgust",
    "Fear",
    "Happy",
    "Sad",
    "Surprise",
    "Neutral",
]
N_CLASSES = len(CLASS_NAMES)
TARGET_COLUMN = "expression"

# Set this to True only when testing the full pipeline quickly.
# Keep it False for the final experiment.
FAST_MODE = False

if FAST_MODE:
    HEAD_EPOCHS = 2
    FINE_TUNE_EPOCHS = 3
    print(
        "FAST_MODE is ON. This is a quick pipeline test, not the final experiment."
    )
else:
    print(
        "FAST_MODE is OFF. Full training settings are active."
    )

tf.keras.utils.set_random_seed(SEED)

raw["label"] = raw["expression"].astype(int)

if raw["label"].isna().any() or not raw["label"].between(0, N_CLASSES - 1).all():
    raise ValueError("Some labels do not match the seven ExpW classes.")

raw["label"] = raw["label"].astype(int)

print("Class order:", dict(enumerate(CLASS_NAMES)))

# These rules are set before model training and before seeing conversion results.
MOBILE_IMAGES_PER_CLASS = 64

FP32_LIMITS = {
    "max_abs": 0.02,
    "mean_abs": 0.002,
    "min_agreement": 0.99,
    "max_balanced_accuracy_drop": 0.01,
    "max_class_recall_drop": 0.03,
}

INT8_LIMITS = {
    "max_abs": 0.15,
    "mean_abs": 0.03,
    "min_agreement": 0.96,

    "max_balanced_accuracy_drop": 0.03,
    "max_class_recall_drop": 0.05,
}

FAST_MODE is OFF. Full training settings are active.
Class order: {0: 'Angry', 1: 'Disgust', 2: 'Fear', 3: 'Happy', 4: 'Sad', 5: 'Surprise', 6:
'Neutral'}

```

8. Use the full ExpW dataset

In this cell we keep all usable ExpW images. The dataset is already close to our target size, so we do not throw away good training images.

Code cell 17

```

# cell:sample
print("Labelled ExpW faces before image checks:", len(raw))
print("We keep the full dataset and remove only bad files or exact repeats later.")

class_table = (
    raw["label"]

```

```

        .value_counts()
        .sort_index()
        .rename("images")
        .to_frame()
    )
class_table["expression"] = [CLASS_NAMES[i] for i in class_table.index]
display(class_table[["expression", "images"]])

Labelled ExpW faces before image checks: 91793
We keep the full dataset and remove only bad files or exact repeats later.

```

	expression	images
label		
0	Angry	3671
1	Disgust	3995
2	Fear	1088
3	Happy	30537
4	Sad	10559
5	Surprise	7060
6	Neutral	34883

9. Check the images

In this cell we open each image and check its face crop. We record bad files instead of hiding them. The check can take a few minutes.

Code cell 19

```

# cell:audit
BOX_COLUMNS = ["face_x", "face_y", "face_width", "face_height"]

def open_face(path, box):
    with Image.open(path) as original:
        original.load()
        image = original.convert("RGB")
    x, y, width, height = map(float, box)
    if not (x == y == width == height == -1):
        if not np.isfinite([x, y, width, height]).all() or width <= 0 or height <= 0:
            raise ValueError("Invalid face box.")
        bounds = (max(0, int(x)), max(0, int(y)),
                  min(image.width, int(x + width)), min(image.height, int(y + height)))
        if bounds[2] <= bounds[0] or bounds[3] <= bounds[1]:
            raise ValueError("Face box is outside the image.")
        image = image.crop(bounds)
    if min(image.size) < 16:
        raise ValueError("Face crop is smaller than 16 pixels.")
    return image

def check_image(row):
    try:
        image = open_face(row["path"], [row[column] for column in BOX_COLUMNS])
        digest = hashlib.sha256(str(image.size).encode() + image.tobytes()).hexdigest()
        return {"sha256": digest, "image_error": ""}
    except Exception as error:
        return {"sha256": "", "image_error": str(error)}

with ThreadPoolExecutor(max_workers=min(8, os.cpu_count() or 1)) as pool:
    checks = list(tqdm(pool.map(check_image, raw.to_dict("records")),
                        total=len(raw), desc="Checking face images"))
checked = pd.concat([raw.reset_index(drop=True), pd.DataFrame(checks)], axis=1)

# Make sure the results folder still exists before saving.
OUTPUT_DIR.mkdir(parents=True, exist_ok=True)

image_check_file = OUTPUT_DIR / "image_checks.csv"
checked.to_csv(image_check_file, index=False)

print("Image check saved to:", image_check_file)
print("Unreadable or invalid images:", int(checked["image_error"].ne("").sum()))
clean_data = checked.loc[checked["image_error"].eq("")]
if clean_data.empty:
    raise ValueError("No usable face images were found.")

```

Checking face images: 100%|#####| 91793/91793 [05:48<00:00, 263.47it/s]

Image check saved to: /content/project_edge_ai/results/expression/20260915_092434/image_checks.csv
Unreadable or invalid images: 0

10. Remove exact repeats

In this cell we remove exact duplicate face images and any exact image that has conflicting labels. Different photos of the same person may still remain.

Code cell 21

```
# cell:duplicates
OUTPUT_DIR.mkdir(parents=True, exist_ok=True)
label_columns = ["label"]

conflicts = clean_data.groupby("sha256")[label_columns].nunique().max(axis=1)
conflict_hashes = set(conflicts[conflicts > 1].index)

conflicting = clean_data["sha256"].isin(conflict_hashes)
print("Images with conflicting labels:", int(conflicting.sum()))

clean_data = clean_data.loc[~conflicting].copy()
clean_data = clean_data.sort_values("path")

repeats = clean_data.duplicated("sha256", keep="first")
clean_data.loc[repeats].to_csv(OUTPUT_DIR / "duplicate_images.csv", index=False)

print("Exact repeated images removed:", int(repeats.sum()))

clean_data = clean_data.loc[~repeats].copy().reset_index(drop=True)

Images with conflicting labels: 94
Exact repeated images removed: 3256
```

11. Separate training, validation and testing

In this cell we make about 80% training, 10% validation and 10% holdout data. Faces from the same original photograph always stay together in one split.

Code cell 23

```
# cell:split
OUTPUT_DIR.mkdir(parents=True, exist_ok=True)

if clean_data["label"].value_counts().min() < 10:
    raise ValueError("A class has too few images for a reliable split.")

# ExpW can contain more than one labelled face from the same photograph.
# Every row with the same original image path must stay in the same split.
groups = clean_data["path"].astype(str).to_numpy()
labels_for_split = clean_data["label"].to_numpy(dtype=np.int32)

splitter = StratifiedGroupKFold(
    n_splits=10,
    shuffle=True,
    random_state=SEED,
)

fold_id = np.full(len(clean_data), -1, dtype=np.int32)

for fold, (_, held_index) in enumerate(
    splitter.split(clean_data, labels_for_split, groups)
):
    fold_id[held_index] = fold

if (fold_id < 0).any():
    raise RuntimeError("Some ExpW rows were not assigned to a split.")

test_df = clean_data.loc[fold_id == 0].copy()
val_df = clean_data.loc[fold_id == 1].copy()
train_df = clean_data.loc[fold_id >= 2].copy()
```

```

train_df = train_df.reset_index(drop=True)
val_df = val_df.reset_index(drop=True)
test_df = test_df.reset_index(drop=True)

frames = {
    "train": train_df,
    "validation": val_df,
    "holdout": test_df,
}

for name, frame in frames.items():
    if set(frame["label"]) != set(range(N_CLASSES)):
        raise ValueError(
            f"The {name} set does not contain every expression class."
        )

    frame.to_csv(
        OUTPUT_DIR / f"{name}_images.csv",
        index=False,
    )

# Exact image data and original photographs must not cross splits.
for first_name, second_name in [
    ("train", "validation"),
    ("train", "holdout"),
    ("validation", "holdout"),
]:
    first = frames[first_name]
    second = frames[second_name]

    if not set(first["sha256"]).isdisjoint(set(second["sha256"])):
        raise RuntimeError(
            f"Exact duplicate leakage between {first_name} and {second_name}."
        )

    if not set(first["path"]).isdisjoint(set(second["path"])):
        raise RuntimeError(
            f"Original photograph leakage between {first_name} and {second_name}."
        )

print({
    name: len(frame)
    for name, frame in frames.items()
})

print(
    "Training percentage:",
    round(100 * len(train_df) / len(clean_data), 2),
)
print(
    "Validation percentage:",
    round(100 * len(val_df) / len(clean_data), 2),
)
print(
    "Holdout percentage:",
    round(100 * len(test_df) / len(clean_data), 2),
)

print("Group leakage check: PASSED")

# Optional small subset for a quick Colab pipeline test.
# This is applied only after the group-safe split.
if FAST_MODE:
    def quick_subset(frame, per_class):
        parts = []

        for label, group in frame.groupby(
            "label"
        ):
            parts.append(
                group.sample(
                    min(
                        per_class,
                        len(group),
                    ),
                ),
            )

```

```

        random_state=(
            SEED
            + int(label)
        ),
    )
)

return pd.concat(
    parts,
    ignore_index=True,
)

train_df = quick_subset(
    train_df,
    1500,
)

val_df = quick_subset(
    val_df,
    250,
)

test_df = quick_subset(
    test_df,
    250,
)

frames = {
    "train": train_df,
    "validation": val_df,
    "holdout": test_df,
}

print(
    "FAST_MODE subset sizes:",
    {
        name: len(frame)
        for name, frame in frames.items()
    },
)

{'train': 70892, 'validation': 8736, 'holdout': 8815}
Training percentage: 80.16
Validation percentage: 9.88
Holdout percentage: 9.97
Group leakage check: PASSED

```

12. Count each class

In this cell we count the images in each class. This shows whether some classes have much less training data.

Code cell 25

```

# cell:counts
OUTPUT_DIR.mkdir(parents=True, exist_ok=True)
counts = pd.DataFrame({name: frame["label"].value_counts().reindex(range(N_CLASSES), fill_value=0)
                        for name, frame in frames.items()})
counts.index = CLASS_NAMES
display(counts)
counts.to_csv(OUTPUT_DIR / "class_counts.csv")
fig, ax = plt.subplots(figsize=(9, 4))
counts["train"].plot.bar(ax=ax)
ax.set(title="Training images per class", xlabel="Class", ylabel="Images")
fig.tight_layout()
fig.savefig(OUTPUT_DIR / "class_counts.png", dpi=150)
plt.show()

```

	train	validation	holdout
Angry	2864	351	375
Disgust	3026	374	393
Fear	843	106	107
Happy	23186	2801	2874
Sad	8315	1047	1043

Surprise	5514	682	663
Neutral	27144	3375	3360



13. Look at a few training images

In this cell we inspect a few face crops and labels. Keep these research images private. Set `SHOW_FACE_IMAGES` to `False` before making a copy that must not contain dataset photos.

Code cell 27

```
# cell:samples
SHOW_FACE_IMAGES = True
if SHOW_FACE_IMAGES:
    for _, row in train_df.sample(3, random_state=SEED).iterrows():
        face = open_face(row["path"], row[BOX_COLUMNS])
        fig, ax = plt.subplots(figsize=(3, 3))
        ax.imshow(face)
        ax.set_title(CLASS_NAMES[int(row["label"])])
        ax.axis("off")
        plt.show()
```

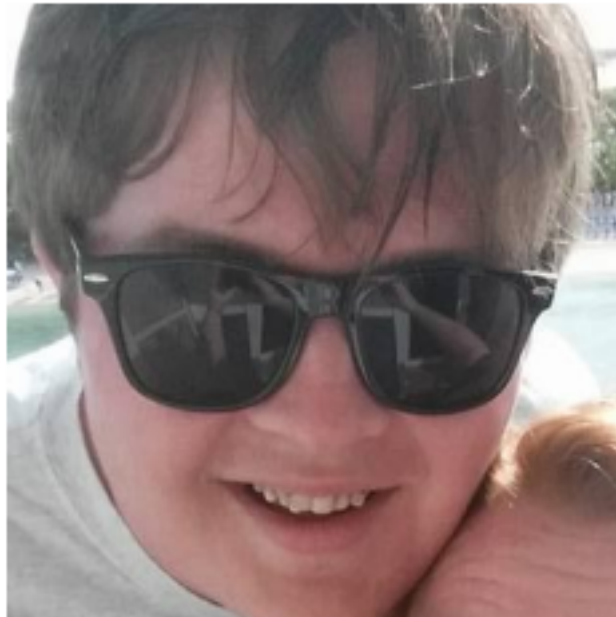
Neutral



Happy



Happy



14. Prepare the model input

In this cell we read small batches of face images. Each image is RGB and 224 by 224 pixels. Pixel values stay from 0 to 255 because input scaling is already inside the model (TensorFlow, n.d.-b).

Code cell 29

```
# cell:input
def load_pixels(path, box):
    # Pillow applies the same face crop for image checks and model input.
    path = path.item() if isinstance(path, np.ndarray) else path
    path = path.decode("utf-8") if isinstance(path, bytes) else str(path)
    image = open_face(path, box).resize((IMAGE_SIZE, IMAGE_SIZE), Image.Resampling.BILINEAR)
    return np.asarray(image, dtype=np.float32)

def read_image(path, box, label):
```

```

image = tf.numpy_function(load_pixels, [path, box], tf.float32)
image.set_shape((IMAGE_SIZE, IMAGE_SIZE, 3))
return image, label

def make_dataset(frame, training=False):
    dataset = tf.data.Dataset.from_tensor_slices((
        frame["path"].astype(str).to_numpy(),
        frame[BOX_COLUMNS].to_numpy(dtype=np.float32),
        frame["label"].to_numpy(dtype=np.int32),
    ))
    if training:
        dataset = dataset.shuffle(min(len(frame), 10000), seed=SEED)
        dataset = dataset.map(read_image, num_parallel_calls=tf.data.AUTOTUNE)
    return dataset.batch(BATCH_SIZE).prefetch(tf.data.AUTOTUNE)

train_ds = make_dataset(train_df, training=True)
val_ds = make_dataset(val_df)
test_ds = make_dataset(test_df)
images, labels = next(iter(train_ds))
print("Batch shape:", images.shape, "Pixel range:", float(tf.reduce_min(images)),
      "to", float(tf.reduce_max(images)))

Batch shape: (128, 224, 224, 3) Pixel range: 0.0 to 255.0

```

15. Make small changes to training images

In this cell we make small random changes to training images. We use horizontal flips, brightness and contrast changes. The function is compiled once so it does not create repeated TensorFlow warnings.

Code cell 31

```

# cell:augmentation
# This function is traced once and reused by the training dataset.
# It avoids repeated Keras Random* conversion and retracing warnings.

@tf.function(
    input_signature=[
        tf.TensorSpec(
            shape=[None, IMAGE_SIZE, IMAGE_SIZE, 3],
            dtype=tf.float32,
        ),
        tf.TensorSpec(
            shape=[None],
            dtype=tf.int32,
        ),
    ],
    reduce_retracing=True,
)
def augment_batch(images, labels):
    images = tf.image.random_flip_left_right(
        images,
        seed=SEED,
    )

    images = tf.image.random_brightness(
        images,
        max_delta=10.0,
        seed=SEED + 1,
    )

    images = tf.image.random_contrast(
        images,
        lower=0.90,
        upper=1.10,
        seed=SEED + 2,
    )

    images = tf.clip_by_value(
        images,
        0.0,
        255.0,
    )

```

```

    return images, labels

train_ds = (
    train_ds
    .map(
        augment_batch,
        num_parallel_calls=tf.data.AUTOTUNE,
    )
    .prefetch(tf.data.AUTOTUNE)
)

print(
    "Training augmentation ready: horizontal flip, "
    "brightness and contrast."
)

Training augmentation ready: horizontal flip, brightness and contrast.

```

16. Give smaller classes more weight

In this cell we give smaller training classes more weight in the loss. The weights use training data only. We cap them so a very small class does not dominate training.

Code cell 33

```

# cell:weights
train_counts = train_df["label"].value_counts().sort_index()
weights = np.sqrt(len(train_df) / (N_CLASSES * train_counts.to_numpy()))
weights = np.minimum(weights, 3.0)
weights /= np.average(weights, weights=train_counts.to_numpy())
CLASS_WEIGHTS = {int(label): float(weight) for label, weight in zip(train_counts.index, weights)}
print("Class weights:", CLASS_WEIGHTS)

Class weights: {0: 2.1500252609025083, 1: 2.0916817514945723, 2: 3.43006010029436, 3:
0.7556438525996397, 4: 1.2618244622486356, 5: 1.5495182242884105, 6: 0.6983821746796912}

```

17. Build the model

In this cell we build EfficientNetV2B0. We start with ImageNet weights and add our own output layer. The image-feature layers stay frozen at first. Model source: Tan and Le (2021).

Code cell 35

```

# cell:model
if not globals().get("GPU_READY", False):
    raise RuntimeError("Pass the GPU test before building the model.")

def build_expression_model(weights="imagenet"):
    backbone = tf.keras.applications.EfficientNetV2B0(
        input_shape=(IMAGE_SIZE, IMAGE_SIZE, 3),
        include_top=False,
        weights=weights,
        include_preprocessing=True,
    )

    backbone.trainable = False

    inputs = tf.keras.Input(
        shape=(IMAGE_SIZE, IMAGE_SIZE, 3),
        name="face_rgb",
    )

    features = backbone(
        inputs,
        training=False,
    )

    features = tf.keras.layers.GlobalAveragePooling2D()(features)
    features = tf.keras.layers.Dropout(0.35)(features)

```



```

        dtype=tf.float32,
        **kwargs,
    )

    if class_id is not None and class_id not in range(N_CLASSES):
        raise ValueError("class_id is outside the expression class range.")

    self.class_id = class_id

    self.cm = self.add_weight(
        name="counts",
        shape=(N_CLASSES, N_CLASSES),
        initializer="zeros",
        dtype=tf.float32,
    )

def update_state(
    self,
    y_true,
    y_pred,
    sample_weight=None,
):
    # Class weights are used for the loss only.
    # These recall values stay unweighted and easy to interpret.
    truth = tf.cast(
        tf.reshape(y_true, [-1]),
        tf.int32,
    )

    prediction = tf.argmax(
        y_pred,
        axis=-1,
        output_type=tf.int32,
    )

    counts = tf.math.confusion_matrix(
        truth,
        prediction,
        num_classes=N_CLASSES,
        dtype=tf.float32,
    )

    self.cm.assign_add(counts)

def result(self):
    recalls = tf.math.divide_no_nan(
        tf.linalg.diag_part(self.cm),
        tf.reduce_sum(self.cm, axis=1),
    )

    if self.class_id is None:
        return tf.reduce_mean(recalls)

    return recalls[self.class_id]

def reset_state(self):
    self.cm.assign(
        tf.zeros_like(self.cm)
    )

def get_config(self):
    return dict(
        super().get_config(),
        class_id=self.class_id,
    )

RECALL_NAMES = [
    "recall_angry",
    "recall_disgust",
    "recall_fear",
    "recall_happy",
    "recall_sad",
    "recall_surprise",

```

```

    "recall_neutral",
]

def compile_model(network, learning_rate):
    metrics = ["accuracy"]

    metrics.extend(
        ExpressionRecall(
            class_id=index,
            name=RECALL_NAMES[index],
        )
        for index in range(N_CLASSES)
    )

    metrics.append(
        ExpressionRecall(
            name="balanced_accuracy"
        )
    )

    network.compile(
        optimizer=tf.keras.optimizers.AdamW(
            learning_rate=learning_rate,
            weight_decay=1e-4,
        ),
        loss="sparse_categorical_crossentropy",
        metrics=metrics,
        jit_compile=False,
    )

class PrintExpressionRecall(tf.keras.callbacks.Callback):
    def on_epoch_end(self, epoch, logs=None):
        values = logs or {}

        parts = []

        for class_name, metric_name in zip(
            CLASS_NAMES,
            RECALL_NAMES,
        ):
            parts.append(
                f"{class_name}: "
                f"{values.get('val_' + metric_name, float('nan')):.1%}"
            )

        print(
            "Validation recall - "
            + " | ".join(parts)
            + f" | Balanced: "
            f"{values.get('val_balanced_accuracy', float('nan')):.1%}",
            flush=True,
        )

def callbacks(stage):
    OUTPUT_DIR.mkdir(
        parents=True,
        exist_ok=True,
    )

    return [
        tf.keras.callbacks.ModelCheckpoint(
            str(
                OUTPUT_DIR
                / f"best_{stage}.weights.h5"
            ),
            monitor="val_balanced_accuracy",
            mode="max",
            save_best_only=True,
            save_weights_only=True,
        ),
        tf.keras.callbacks.EarlyStopping(
            monitor="val_balanced_accuracy",
            mode="max",
            patience=5,
        ),
    ]

```

```

        restore_best_weights=True,
    ),
    tf.keras.callbacks.ReduceLROnPlateau(
        monitor="val_balanced_accuracy",
        mode="max",
        patience=2,
        factor=0.5,
        min_lr=1e-7,
    ),
    tf.keras.callbacks.CSVLogger(
        str(
            OUTPUT_DIR
            / f"{stage}_training.csv"
        )
    ),
    PrintExpressionRecall(),
    tf.keras.callbacks.TerminateOnNaN(),
]

compile_model(
    model,
    1e-3,
)

print(
    "Each epoch reports validation recall for all seven "
    "expressions and balanced accuracy."
)
print(
    "The best checkpoint is selected by validation "
    "balanced accuracy."
)

```

Each epoch reports validation recall for all seven expressions and balanced accuracy.
The best checkpoint is selected by validation balanced accuracy.

19. Train the new output layer

In this cell we train the model's new output layer. The pretrained image-feature layers do not change yet.

Code cell 39

```

# cell:train_head
if not globals().get("GPU_READY", False):
    raise RuntimeError("GPU check has not passed. Training is stopped.")

started = time.perf_counter()

head_history = model.fit(
    train_ds,
    validation_data=val_ds,
    epochs=HEAD_EPOCHS,
    class_weight=CLASS_WEIGHTS,
    callbacks=callbacks("head"),
    verbose=2,
)

head_seconds = time.perf_counter() - started

print(
    "First training stage, minutes:",
    round(head_seconds / 60, 2),
)

Epoch 1/8
Validation recall - Angry: 10.8% | Disgust: 0.8% | Fear: 2.8% | Happy: 41.6% | Sad: 26.5% |
Surprise: 55.7% | Neutral: 78.3% | Balanced: 30.9%
554/554 - 560s - 1s/step - accuracy: 0.4985 - balanced_accuracy: 0.3022 - loss: 1.5718 -
recall_angry: 0.1892 - recall_disgust: 0.0218 - recall_fear: 0.0415 - recall_happy: 0.6208 -
recall_neutral: 0.5731 - recall_sad: 0.3763 - recall_surprise: 0.2929 - val_accuracy: 0.5164 -
val_balanced_accuracy: 0.3094 - val_loss: 1.4020 - val_recall_angry: 0.1083 - val_recall_disgust:
0.0080 - val_recall_fear: 0.0283 - val_recall_happy: 0.4163 - val_recall_neutral: 0.7834 -

```


val_recall_sad: 0.2646 - val_recall_surprise: 0.5572 - learning_rate: 0.0010
Epoch 2/8
Validation recall - Angry: 16.5% | Disgust: 2.4% | Fear: 7.5% | Happy: 49.3% | Sad: 31.7% | Surprise: 48.7% | Neutral: 78.1% | Balanced: 33.5%
554/554 - 512s - 925ms/step - accuracy: 0.5329 - balanced_accuracy: 0.3411 - loss: 1.4820 - recall_angry: 0.2654 - recall_disgust: 0.0248 - recall_fear: 0.0712 - recall_happy: 0.6385 - recall_neutral: 0.6148 - recall_sad: 0.4029 - recall_surprise: 0.3701 - val_accuracy: 0.5444 - val_balanced_accuracy: 0.3347 - val_loss: 1.3497 - val_recall_angry: 0.1652 - val_recall_disgust: 0.0241 - val_recall_fear: 0.0755 - val_recall_happy: 0.4934 - val_recall_neutral: 0.7807 - val_recall_sad: 0.3171 - val_recall_surprise: 0.4868 - learning_rate: 0.0010
Epoch 3/8
Validation recall - Angry: 23.9% | Disgust: 0.5% | Fear: 8.5% | Happy: 50.2% | Sad: 31.9% | Surprise: 50.7% | Neutral: 78.3% | Balanced: 34.9%
554/554 - 548s - 990ms/step - accuracy: 0.5398 - balanced_accuracy: 0.3520 - loss: 1.4574 - recall_angry: 0.2940 - recall_disgust: 0.0307 - recall_fear: 0.0937 - recall_happy: 0.6486 - recall_neutral: 0.6190 - recall_sad: 0.4000 - recall_surprise: 0.3781 - val_accuracy: 0.5522 - val_balanced_accuracy: 0.3487 - val_loss: 1.3339 - val_recall_angry: 0.2393 - val_recall_disgust: 0.0053 - val_recall_fear: 0.0849 - val_recall_happy: 0.5023 - val_recall_neutral: 0.7828 - val_recall_sad: 0.3190 - val_recall_surprise: 0.5073 - learning_rate: 0.0010
Epoch 4/8
Validation recall - Angry: 19.4% | Disgust: 2.1% | Fear: 2.8% | Happy: 49.1% | Sad: 29.8% | Surprise: 59.1% | Neutral: 77.1% | Balanced: 34.2%
554/554 - 553s - 999ms/step - accuracy: 0.5410 - balanced_accuracy: 0.3543 - loss: 1.4523 - recall_angry: 0.2950 - recall_disgust: 0.0330 - recall_fear: 0.0937 - recall_happy: 0.6493 - recall_neutral: 0.6194 - recall_sad: 0.3945 - recall_surprise: 0.3952 - val_accuracy: 0.5461 - val_balanced_accuracy: 0.3420 - val_loss: 1.3415 - val_recall_angry: 0.1937 - val_recall_disgust: 0.0214 - val_recall_fear: 0.0283 - val_recall_happy: 0.4909 - val_recall_neutral: 0.7710 - val_recall_sad: 0.2980 - val_recall_surprise: 0.5909 - learning_rate: 0.0010
Epoch 5/8
Validation recall - Angry: 16.0% | Disgust: 2.7% | Fear: 5.7% | Happy: 53.2% | Sad: 31.6% | Surprise: 54.0% | Neutral: 77.4% | Balanced: 34.4%
554/554 - 535s - 966ms/step - accuracy: 0.5425 - balanced_accuracy: 0.3617 - loss: 1.4428 - recall_angry: 0.3059 - recall_disgust: 0.0373 - recall_fear: 0.1139 - recall_happy: 0.6463 - recall_neutral: 0.6186 - recall_sad: 0.4055 - recall_surprise: 0.4041 - val_accuracy: 0.5578 - val_balanced_accuracy: 0.3435 - val_loss: 1.2916 - val_recall_angry: 0.1595 - val_recall_disgust: 0.0267 - val_recall_fear: 0.0566 - val_recall_happy: 0.5323 - val_recall_neutral: 0.7736 - val_recall_sad: 0.3161 - val_recall_surprise: 0.5396 - learning_rate: 0.0010
Epoch 6/8
Validation recall - Angry: 25.1% | Disgust: 2.4% | Fear: 4.7% | Happy: 50.0% | Sad: 31.2% | Surprise: 52.2% | Neutral: 81.2% | Balanced: 35.3%
554/554 - 559s - 1s/step - accuracy: 0.5429 - balanced_accuracy: 0.3614 - loss: 1.4374 - recall_angry: 0.3094 - recall_disgust: 0.0314 - recall_fear: 0.1151 - recall_happy: 0.6466 - recall_neutral: 0.6204 - recall_sad: 0.4010 - recall_surprise: 0.4059 - val_accuracy: 0.5639 - val_balanced_accuracy: 0.3526 - val_loss: 1.2967 - val_recall_angry: 0.2507 - val_recall_disgust: 0.0241 - val_recall_fear: 0.0472 - val_recall_happy: 0.5002 - val_recall_neutral: 0.8119 - val_recall_sad: 0.3123 - val_recall_surprise: 0.5220 - learning_rate: 5.0000e-04
Epoch 7/8
Validation recall - Angry: 22.2% | Disgust: 1.3% | Fear: 11.3% | Happy: 53.8% | Sad: 27.4% | Surprise: 50.4% | Neutral: 80.7% | Balanced: 35.3%
554/554 - 506s - 913ms/step - accuracy: 0.5453 - balanced_accuracy: 0.3634 - loss: 1.4314 - recall_angry: 0.3177 - recall_disgust: 0.0321 - recall_fear: 0.1091 - recall_happy: 0.6452 - recall_neutral: 0.6250 - recall_sad: 0.4070 - recall_surprise: 0.4079 - val_accuracy: 0.5675 - val_balanced_accuracy: 0.3532 - val_loss: 1.2829 - val_recall_angry: 0.2222 - val_recall_disgust: 0.0134 - val_recall_fear: 0.1132 - val_recall_happy: 0.5380 - val_recall_neutral: 0.8074 - val_recall_sad: 0.2741 - val_recall_surprise: 0.5044 - learning_rate: 5.0000e-04
Epoch 8/8
Validation recall - Angry: 19.1% | Disgust: 3.5% | Fear: 5.7% | Happy: 51.6% | Sad: 33.7% | Surprise: 54.7% | Neutral: 78.1% | Balanced: 35.2%
554/554 - 517s - 933ms/step - accuracy: 0.5445 - balanced_accuracy: 0.3618 - loss: 1.4330 - recall_angry: 0.3142 - recall_disgust: 0.0330 - recall_fear: 0.1056 - recall_happy: 0.6472 - recall_neutral: 0.6231 - recall_sad: 0.4030 - recall_surprise: 0.4062 - val_accuracy: 0.5601 - val_balanced_accuracy: 0.3519 - val_loss: 1.3059 - val_recall_angry: 0.1909 - val_recall_disgust: 0.0348 - val_recall_fear: 0.0566 - val_recall_happy: 0.5155 - val_recall_neutral: 0.7813 - val_recall_sad: 0.3372 - val_recall_surprise: 0.5469 - learning_rate: 5.0000e-04
First training stage, minutes: 71.54

20. Prepare fine-tuning

In this cell we load the best first-stage model. We allow the last third of the feature layers to learn. Batch-normalisation layers stay frozen.

Code cell 41

```
# cell:unfreeze
OUTPUT_DIR.mkdir(
    parents=True,
    exist_ok=True,
)

model.load_weights(
    str(
        OUTPUT_DIR
        / "best_head.weights.h5"
    )
)

backbone = model.get_layer(
    BACKBONE_NAME
)

backbone.trainable = True

cutoff = int(
    len(backbone.layers) * 2 / 3
)

for index, layer in enumerate(
    backbone.layers
):
    layer.trainable = (
        index >= cutoff
        and not isinstance(
            layer,
            tf.keras.layers.BatchNormalization,
        )
    )

compile_model(
    model,
    1e-5,
)

print(
    "Trainable feature layers:",
    sum(
        layer.trainable
        for layer in backbone.layers
    ),
)

Trainable feature layers: 72
```

21. Fine-tune the model

In this cell we train again with a smaller learning rate. The output layer and selected feature layers learn together.

Code cell 43

```
# cell:train_fine
if not globals().get("GPU_READY", False):
    raise RuntimeError("GPU check has not passed. Training is stopped.")

started = time.perf_counter()

fine_history = model.fit(
    train_ds,
    validation_data=val_ds,
    epochs=FINE_TUNE_EPOCHS,
    class_weight=CLASS_WEIGHTS,
    callbacks=callbacks("fine"),
    verbose=2,
)

fine_seconds = time.perf_counter() - started
```

```

print(
    "Fine-tuning, minutes:",
    round(fine_seconds / 60, 2),
)

Epoch 1/20
Validation recall - Angry: 28.8% | Disgust: 1.6% | Fear: 7.5% | Happy: 57.1% | Sad: 32.9% |
Surprise: 56.0% | Neutral: 81.0% | Balanced: 37.8%
554/554 - 570s - 1s/step - accuracy: 0.5596 - balanced_accuracy: 0.3723 - loss: 1.4020 -
recall_angry: 0.3027 - recall_disgust: 0.0288 - recall_fear: 0.1388 - recall_happy: 0.6574 -
recall_neutral: 0.6528 - recall_sad: 0.3864 - recall_surprise: 0.4392 - val_accuracy: 0.5921 -
val_balanced_accuracy: 0.3784 - val_loss: 1.2266 - val_recall_angry: 0.2877 - val_recall_disgust:
0.0160 - val_recall_fear: 0.0755 - val_recall_happy: 0.5705 - val_recall_neutral: 0.8101 -
val_recall_sad: 0.3286 - val_recall_surprise: 0.5601 - learning_rate: 1.0000e-05
Epoch 2/20
Validation recall - Angry: 32.2% | Disgust: 2.1% | Fear: 6.6% | Happy: 61.0% | Sad: 33.3% |
Surprise: 58.7% | Neutral: 80.4% | Balanced: 39.2%
554/554 - 518s - 935ms/step - accuracy: 0.5865 - balanced_accuracy: 0.3923 - loss: 1.3503 -
recall_angry: 0.3324 - recall_disgust: 0.0297 - recall_fear: 0.1400 - recall_happy: 0.6783 -
recall_neutral: 0.6897 - recall_sad: 0.4053 - recall_surprise: 0.4710 - val_accuracy: 0.6069 -
val_balanced_accuracy: 0.3920 - val_loss: 1.1818 - val_recall_angry: 0.3219 - val_recall_disgust:
0.0214 - val_recall_fear: 0.0660 - val_recall_happy: 0.6105 - val_recall_neutral: 0.8044 -
val_recall_sad: 0.3333 - val_recall_surprise: 0.5865 - learning_rate: 1.0000e-05
Epoch 3/20
Validation recall - Angry: 33.3% | Disgust: 1.3% | Fear: 8.5% | Happy: 63.8% | Sad: 33.7% |
Surprise: 57.8% | Neutral: 81.2% | Balanced: 40.0%
554/554 - 523s - 944ms/step - accuracy: 0.5957 - balanced_accuracy: 0.3983 - loss: 1.3191 -
recall_angry: 0.3547 - recall_disgust: 0.0245 - recall_fear: 0.1257 - recall_happy: 0.6864 -
recall_neutral: 0.7020 - recall_sad: 0.4042 - recall_surprise: 0.4909 - val_accuracy: 0.6188 -
val_balanced_accuracy: 0.3995 - val_loss: 1.1398 - val_recall_angry: 0.3333 - val_recall_disgust:
0.0134 - val_recall_fear: 0.0849 - val_recall_happy: 0.6383 - val_recall_neutral: 0.8119 -
val_recall_sad: 0.3372 - val_recall_surprise: 0.5777 - learning_rate: 1.0000e-05
Epoch 4/20
Validation recall - Angry: 35.6% | Disgust: 1.9% | Fear: 9.4% | Happy: 64.7% | Sad: 37.3% |
Surprise: 60.6% | Neutral: 79.5% | Balanced: 41.3%

554/554 - 570s - 1s/step - accuracy: 0.6102 - balanced_accuracy: 0.4120 - loss: 1.2873 -
recall_angry: 0.3816 - recall_disgust: 0.0307 - recall_fear: 0.1352 - recall_happy: 0.7014 -
recall_neutral: 0.7169 - recall_sad: 0.4177 - recall_surprise: 0.5004 - val_accuracy: 0.6227 -
val_balanced_accuracy: 0.4128 - val_loss: 1.1243 - val_recall_angry: 0.3561 - val_recall_disgust:
0.0187 - val_recall_fear: 0.0943 - val_recall_happy: 0.6466 - val_recall_neutral: 0.7950 -
val_recall_sad: 0.3734 - val_recall_surprise: 0.6056 - learning_rate: 1.0000e-05
Epoch 5/20
Validation recall - Angry: 36.8% | Disgust: 1.3% | Fear: 9.4% | Happy: 63.9% | Sad: 36.0% |
Surprise: 59.8% | Neutral: 82.6% | Balanced: 41.4%
554/554 - 532s - 961ms/step - accuracy: 0.6162 - balanced_accuracy: 0.4204 - loss: 1.2642 -
recall_angry: 0.3928 - recall_disgust: 0.0370 - recall_fear: 0.1412 - recall_happy: 0.7021 -
recall_neutral: 0.7235 - recall_sad: 0.4249 - recall_surprise: 0.5210 - val_accuracy: 0.6303 -
val_balanced_accuracy: 0.4140 - val_loss: 1.1004 - val_recall_angry: 0.3675 - val_recall_disgust:
0.0134 - val_recall_fear: 0.0943 - val_recall_happy: 0.6387 - val_recall_neutral: 0.8261 -
val_recall_sad: 0.3601 - val_recall_surprise: 0.5982 - learning_rate: 1.0000e-05
Epoch 6/20
Validation recall - Angry: 38.7% | Disgust: 3.7% | Fear: 11.3% | Happy: 66.4% | Sad: 37.2% |
Surprise: 59.5% | Neutral: 81.3% | Balanced: 42.6%
554/554 - 573s - 1s/step - accuracy: 0.6259 - balanced_accuracy: 0.4330 - loss: 1.2437 -
recall_angry: 0.4162 - recall_disgust: 0.0420 - recall_fear: 0.1578 - recall_happy: 0.7111 -
recall_neutral: 0.7311 - recall_sad: 0.4361 - recall_surprise: 0.5368 - val_accuracy: 0.6368 -
val_balanced_accuracy: 0.4261 - val_loss: 1.0785 - val_recall_angry: 0.3875 - val_recall_disgust:
0.0374 - val_recall_fear: 0.1132 - val_recall_happy: 0.6644 - val_recall_neutral: 0.8133 -
val_recall_sad: 0.3715 - val_recall_surprise: 0.5953 - learning_rate: 1.0000e-05
Epoch 7/20
Validation recall - Angry: 39.0% | Disgust: 3.7% | Fear: 11.3% | Happy: 64.7% | Sad: 38.9% |
Surprise: 61.1% | Neutral: 82.9% | Balanced: 43.1%
554/554 - 574s - 1s/step - accuracy: 0.6318 - balanced_accuracy: 0.4374 - loss: 1.2247 -
recall_angry: 0.4361 - recall_disgust: 0.0459 - recall_fear: 0.1542 - recall_happy: 0.7222 -
recall_neutral: 0.7362 - recall_sad: 0.4326 - recall_surprise: 0.5343 - val_accuracy: 0.6409 -

val_balanced_accuracy: 0.4311 - val_loss: 1.0692 - val_recall_angry: 0.3903 - val_recall_disgust:
0.0374 - val_recall_fear: 0.1132 - val_recall_happy: 0.6473 - val_recall_neutral: 0.8293 -
val_recall_sad: 0.3887 - val_recall_surprise: 0.6114 - learning_rate: 1.0000e-05
Epoch 8/20
Validation recall - Angry: 39.3% | Disgust: 2.9% | Fear: 12.3% | Happy: 66.7% | Sad: 37.0% |
Surprise: 62.0% | Neutral: 82.2% | Balanced: 43.2%
554/554 - 531s - 958ms/step - accuracy: 0.6350 - balanced_accuracy: 0.4404 - loss: 1.2075 -
recall_angry: 0.4277 - recall_disgust: 0.0440 - recall_fear: 0.1530 - recall_happy: 0.7204 -

```

recall_neutral: 0.7396 - recall_sad: 0.4453 - recall_surprise: 0.5531 - val_accuracy: 0.6424 -
 val_balanced_accuracy: 0.4319 - val_loss: 1.0577 - val_recall_angry: 0.3932 - val_recall_disgust:
 0.0294 - val_recall_fear: 0.1226 - val_recall_happy: 0.6665 - val_recall_neutral: 0.8216 -
 val_recall_sad: 0.3696 - val_recall_surprise: 0.6202 - learning_rate: 1.0000e-05
 Epoch 9/20
 Validation recall - Angry: 41.3% | Disgust: 5.1% | Fear: 14.2% | Happy: 69.3% | Sad: 37.4% |
 Surprise: 62.6% | Neutral: 80.3% | Balanced: 44.3%
 554/554 - 524s - 945ms/step - accuracy: 0.6383 - balanced_accuracy: 0.4430 - loss: 1.1975 -
 recall_angry: 0.4337 - recall_disgust: 0.0426 - recall_fear: 0.1542 - recall_happy: 0.7237 -
 recall_neutral: 0.7439 - recall_sad: 0.4451 - recall_surprise: 0.5577 - val_accuracy: 0.6469 -
 val_balanced_accuracy: 0.4432 - val_loss: 1.0484 - val_recall_angry: 0.4131 - val_recall_disgust:
 0.0508 - val_recall_fear: 0.1415 - val_recall_happy: 0.6933 - val_recall_neutral: 0.8033 -
 val_recall_sad: 0.3744 - val_recall_surprise: 0.6261 - learning_rate: 1.0000e-05
 Epoch 10/20
 Validation recall - Angry: 44.2% | Disgust: 4.3% | Fear: 13.2% | Happy: 68.7% | Sad: 40.2% |
 Surprise: 62.2% | Neutral: 81.4% | Balanced: 44.9%
 554/554 - 578s - 1s/step - accuracy: 0.6437 - balanced_accuracy: 0.4491 - loss: 1.1848 -
 recall_angry: 0.4501 - recall_disgust: 0.0446 - recall_fear: 0.1578 - recall_happy: 0.7305 -
 recall_neutral: 0.7476 - recall_sad: 0.4505 - recall_surprise: 0.5626 - val_accuracy: 0.6528 -
 val_balanced_accuracy: 0.4488 - val_loss: 1.0368 - val_recall_angry: 0.4416 - val_recall_disgust:
 0.0428 - val_recall_fear: 0.1321 - val_recall_happy: 0.6873 - val_recall_neutral: 0.8142 -
 val_recall_sad: 0.4021 - val_recall_surprise: 0.6217 - learning_rate: 1.0000e-05
 Epoch 11/20
 Validation recall - Angry: 41.0% | Disgust: 4.5% | Fear: 14.2% | Happy: 68.5% | Sad: 40.7% |
 Surprise: 64.5% | Neutral: 81.6% | Balanced: 45.0%
 554/554 - 526s - 950ms/step - accuracy: 0.6448 - balanced_accuracy: 0.4534 - loss: 1.1743 -
 recall_angry: 0.4623 - recall_disgust: 0.0506 - recall_fear: 0.1720 - recall_happy: 0.7317 -
 recall_neutral: 0.7483 - recall_sad: 0.4441 - recall_surprise: 0.5647 - val_accuracy: 0.6540 -
 val_balanced_accuracy: 0.4500 - val_loss: 1.0345 - val_recall_angry: 0.4103 - val_recall_disgust:
 0.0455 - val_recall_fear: 0.1415 - val_recall_happy: 0.6848 - val_recall_neutral: 0.8157 -
 val_recall_sad: 0.4069 - val_recall_surprise: 0.6452 - learning_rate: 1.0000e-05
 Epoch 12/20
 Validation recall - Angry: 45.3% | Disgust: 5.6% | Fear: 14.2% | Happy: 69.4% | Sad: 38.5% |
 Surprise: 63.6% | Neutral: 81.2% | Balanced: 45.4%
 554/554 - 575s - 1s/step - accuracy: 0.6507 - balanced_accuracy: 0.4569 - loss: 1.1599 -
 recall_angry: 0.4557 - recall_disgust: 0.0552 - recall_fear: 0.1684 - recall_happy: 0.7407 -
 recall_neutral: 0.7513 - recall_sad: 0.4590 - recall_surprise: 0.5678 - val_accuracy: 0.6546 -
 val_balanced_accuracy: 0.4541 - val_loss: 1.0273 - val_recall_angry: 0.4530 - val_recall_disgust:
 0.0561 - val_recall_fear: 0.1415 - val_recall_happy: 0.6944 - val_recall_neutral: 0.8124 -
 val_recall_sad: 0.3849 - val_recall_surprise: 0.6364 - learning_rate: 1.0000e-05
 Epoch 13/20
 Validation recall - Angry: 45.3% | Disgust: 5.3% | Fear: 14.2% | Happy: 70.5% | Sad: 40.3% |
 Surprise: 62.8% | Neutral: 81.3% | Balanced: 45.7%
 554/554 - 529s - 955ms/step - accuracy: 0.6540 - balanced_accuracy: 0.4646 - loss: 1.1496 -
 recall_angry: 0.4672 - recall_disgust: 0.0661 - recall_fear: 0.1862 - recall_happy: 0.7395 -
 recall_neutral: 0.7552 - recall_sad: 0.4641 - recall_surprise: 0.5740 - val_accuracy: 0.6596 -
 val_balanced_accuracy: 0.4566 - val_loss: 1.0111 - val_recall_angry: 0.4530 - val_recall_disgust:
 0.0535 - val_recall_fear: 0.1415 - val_recall_happy: 0.7051 - val_recall_neutral: 0.8127 -
 val_recall_sad: 0.4031 - val_recall_surprise: 0.6276 - learning_rate: 1.0000e-05
 Epoch 14/20
 Validation recall - Angry: 47.3% | Disgust: 5.6% | Fear: 15.1% | Happy: 72.3% | Sad: 40.1% |
 Surprise: 61.9% | Neutral: 80.4% | Balanced: 46.1%
 554/554 - 554s - 1s/step - accuracy: 0.6553 - balanced_accuracy: 0.4634 - loss: 1.1432 -
 recall_angry: 0.4794 - recall_disgust: 0.0516 - recall_fear: 0.1744 - recall_happy: 0.7422 -
 recall_neutral: 0.7570 - recall_sad: 0.4610 - recall_surprise: 0.5783 - val_accuracy: 0.6622 -
 val_balanced_accuracy: 0.4611 - val_loss: 1.0026 - val_recall_angry: 0.4729 - val_recall_disgust:
 0.0561 - val_recall_fear: 0.1509 - val_recall_happy: 0.7233 - val_recall_neutral: 0.8041 -
 val_recall_sad: 0.4011 - val_recall_surprise: 0.6188 - learning_rate: 1.0000e-05
 Epoch 15/20
 Validation recall - Angry: 45.3% | Disgust: 5.6% | Fear: 13.2% | Happy: 72.6% | Sad: 40.2% |
 Surprise: 63.2% | Neutral: 80.9% | Balanced: 45.9%
 554/554 - 530s - 957ms/step - accuracy: 0.6588 - balanced_accuracy: 0.4718 - loss: 1.1280 -
 recall_angry: 0.4934 - recall_disgust: 0.0628 - recall_fear: 0.1945 - recall_happy: 0.7473 -
 recall_neutral: 0.7553 - recall_sad: 0.4710 - recall_surprise: 0.5785 - val_accuracy: 0.6648 -
 val_balanced_accuracy: 0.4585 - val_loss: 0.9891 - val_recall_angry: 0.4530 - val_recall_disgust:
 0.0561 - val_recall_fear: 0.1321 - val_recall_happy: 0.7258 - val_recall_neutral: 0.8086 -
 val_recall_sad: 0.4021 - val_recall_surprise: 0.6320 - learning_rate: 1.0000e-05
 Epoch 16/20
 Validation recall - Angry: 47.0% | Disgust: 5.9% | Fear: 13.2% | Happy: 72.1% | Sad: 42.2% |
 Surprise: 66.0% | Neutral: 79.3% | Balanced: 46.5%
 554/554 - 513s - 925ms/step - accuracy: 0.6599 - balanced_accuracy: 0.4715 - loss: 1.1238 -
 recall_angry: 0.4867 - recall_disgust: 0.0625 - recall_fear: 0.1934 - recall_happy: 0.7482 -
 recall_neutral: 0.7583 - recall_sad: 0.4666 - recall_surprise: 0.5847 - val_accuracy: 0.6625 -

```

val_balanced_accuracy: 0.4652 - val_loss: 1.0060 - val_recall_angry: 0.4701 - val_recall_disgust:
0.0588 - val_recall_fear: 0.1321 - val_recall_happy: 0.7208 - val_recall_neutral: 0.7929 -
val_recall_sad: 0.4222 - val_recall_surprise: 0.6598 - learning_rate: 1.0000e-05
Epoch 17/20
Validation recall - Angry: 48.7% | Disgust: 6.7% | Fear: 16.0% | Happy: 71.7% | Sad: 42.9% |
Surprise: 64.8% | Neutral: 78.7% | Balanced: 47.1%
554/554 - 514s - 928ms/step - accuracy: 0.6642 - balanced_accuracy: 0.4790 - loss: 1.1141 -
recall_angry: 0.5028 - recall_disgust: 0.0744 - recall_fear: 0.1945 - recall_happy: 0.7520 -
recall_neutral: 0.7583 - recall_sad: 0.4771 - recall_surprise: 0.5936 - val_accuracy: 0.6603 -

val_balanced_accuracy: 0.4707 - val_loss: 1.0117 - val_recall_angry: 0.4872 - val_recall_disgust:
0.0668 - val_recall_fear: 0.1604 - val_recall_happy: 0.7169 - val_recall_neutral: 0.7870 -
val_recall_sad: 0.4288 - val_recall_surprise: 0.6481 - learning_rate: 1.0000e-05
Epoch 18/20
Validation recall - Angry: 45.6% | Disgust: 6.4% | Fear: 15.1% | Happy: 72.1% | Sad: 39.2% |
Surprise: 65.8% | Neutral: 81.5% | Balanced: 46.5%
554/554 - 570s - 1s/step - accuracy: 0.6642 - balanced_accuracy: 0.4811 - loss: 1.1039 -
recall_angry: 0.5007 - recall_disgust: 0.0677 - recall_fear: 0.2183 - recall_happy: 0.7497 -
recall_neutral: 0.7601 - recall_sad: 0.4788 - recall_surprise: 0.5923 - val_accuracy: 0.6672 -
val_balanced_accuracy: 0.4653 - val_loss: 0.9933 - val_recall_angry: 0.4558 - val_recall_disgust:
0.0642 - val_recall_fear: 0.1509 - val_recall_happy: 0.7212 - val_recall_neutral: 0.8148 -
val_recall_sad: 0.3916 - val_recall_surprise: 0.6584 - learning_rate: 1.0000e-05
Epoch 19/20
Validation recall - Angry: 46.7% | Disgust: 6.1% | Fear: 14.2% | Happy: 72.9% | Sad: 39.7% |
Surprise: 66.4% | Neutral: 80.3% | Balanced: 46.6%
554/554 - 514s - 927ms/step - accuracy: 0.6659 - balanced_accuracy: 0.4827 - loss: 1.0978 -
recall_angry: 0.5091 - recall_disgust: 0.0648 - recall_fear: 0.2195 - recall_happy: 0.7539 -
recall_neutral: 0.7606 - recall_sad: 0.4773 - recall_surprise: 0.5938 - val_accuracy: 0.6666 -
val_balanced_accuracy: 0.4663 - val_loss: 0.9852 - val_recall_angry: 0.4672 - val_recall_disgust:
0.0615 - val_recall_fear: 0.1415 - val_recall_happy: 0.7290 - val_recall_neutral: 0.8030 -
val_recall_sad: 0.3973 - val_recall_surprise: 0.6642 - learning_rate: 1.0000e-05
Epoch 20/20
Validation recall - Angry: 46.4% | Disgust: 5.9% | Fear: 14.2% | Happy: 72.3% | Sad: 43.8% |
Surprise: 65.5% | Neutral: 80.8% | Balanced: 47.0%
554/554 - 523s - 944ms/step - accuracy: 0.6687 - balanced_accuracy: 0.4881 - loss: 1.0948 -
recall_angry: 0.5171 - recall_disgust: 0.0744 - recall_fear: 0.2313 - recall_happy: 0.7567 -
recall_neutral: 0.7626 - recall_sad: 0.4760 - recall_surprise: 0.5988 - val_accuracy: 0.6706 -
val_balanced_accuracy: 0.4699 - val_loss: 0.9822 - val_recall_angry: 0.4644 - val_recall_disgust:
0.0588 - val_recall_fear: 0.1415 - val_recall_happy: 0.7230 - val_recall_neutral: 0.8080 -
val_recall_sad: 0.4384 - val_recall_surprise: 0.6554 - learning_rate: 5.0000e-06

Fine-tuning, minutes: 180.69

```

Code cell 44

```

# Find the saved model files and back them up before doing anything else.

from pathlib import Path
from datetime import datetime
import shutil
from google.colab import files as colab_files

RECOVERY_ROOT = Path(
    "/content/project_edge_ai/results/expression"
)

if not RECOVERY_ROOT.is_dir():
    raise FileNotFoundError(
        "The old expression-results folder is not present in this runtime. "
        "Do not start training yet. Check for a downloaded results ZIP "
        "or a Google Drive backup."
    )

# Look for saved weights, complete models and Android exports.
saved_models = sorted(
    path
    for path in RECOVERY_ROOT.rglob("")
    if path.is_file()
    and path.suffix.lower() in {".h5", ".keras", ".tflite"}
    and path.stat().st_size > 0
)

print("SAVED MODEL FILES")
print("-" * 60)

```

```

if saved_models:
    for path in saved_models:
        size_mb = path.stat().st_size / (1024 * 1024)
        print(f"{path}\n Size: {size_mb:.2f} MB\n")
else:
    print("No model files were found in this results folder.")
    print("Any remaining logs will still be included in the backup.")

# Show which Python objects survived the session crash.
print("\nOBJECTS STILL IN MEMORY")
for name in ["OUTPUT_DIR", "model", "compile_model", "val_ds"]:
    status = "present" if name in globals() else "missing"
    print(f"{name}: {status}")

# Back up the results only-not the large image dataset.
backup_name = (
    Path("/content")
    / (
        "expression_recovery_"
        + datetime.now().strftime("%Y%m%d_%H%M%S_%f")
    )
)

backup_zip = shutil.make_archive(
    str(backup_name),
    "zip",
    root_dir=str(RECOVERY_ROOT),
)

print("\nRecovery ZIP created:", backup_zip)

print("Save this ZIP on your laptop.")
print("This cell has not run any training.")

colab_files.download(backup_zip)

SAVED MODEL FILES
-----
/content/project_edge_ai/results/expression/20260915_092434/best_fine.weights.h5
Size: 75.55 MB

/content/project_edge_ai/results/expression/20260915_092434/best_head.weights.h5
Size: 23.43 MB

/content/project_edge_ai/results/expression/20260915_092434/best_model.h5
Size: 23.12 MB

/content/project_edge_ai/results/expression/20260915_092434/expression_fp32.tflite
Size: 22.36 MB

/content/project_edge_ai/results/expression/20260915_092434/expression_int8.tflite
Size: 6.81 MB

OBJECTS STILL IN MEMORY
OUTPUT_DIR: missing
model: missing
compile_model: missing
val_ds: missing

Recovery ZIP created: /content/expression_recovery_20260915_140627_123446.zip
Save this ZIP on your laptop.
This cell has not run any training.

<IPython.core.display.Javascript object>

<IPython.core.display.Javascript object>

```

Code cell 45

```

# Restore the saved expression model. No training is run.

from pathlib import Path
import numpy as np
import pandas as pd

```

```

import tensorflow as tf
from IPython.display import display

PROJECT_DIR = Path("/content/project_edge_ai")

# Use the exact saved run shown in your screenshot.
OUTPUT_DIR = (
    PROJECT_DIR
    / "results"
    / "expression"
    / "20260915_092434"
)

TASK = "expression"
NOTEBOOK_NAME = "03_Expression_Model_Colab"

CLASS_NAMES = [
    "Angry",
    "Disgust",
    "Fear",
    "Happy",
    "Sad",
    "Surprise",
    "Neutral",
]

N_CLASSES = len(CLASS_NAMES)
IMAGE_SIZE = 224

MODEL_PATH = OUTPUT_DIR / "best_model.h5"

if not MODEL_PATH.is_file():
    raise FileNotFoundError(
        f"The saved model is no longer at {MODEL_PATH}. "
        "Keep your recovery ZIP. Do not restart training."
    )

print("Loading the saved model:", MODEL_PATH)

# Load and test on the CPU. This does not need a training GPU.
with tf.device("/CPU:0"):
    recovered_model = tf.keras.models.load_model(
        str(MODEL_PATH),
        compile=False,
    )

    if len(recovered_model.inputs) != 1:
        raise ValueError("Expected one image input.")

    if tuple(recovered_model.input_shape[1:]) != (
        IMAGE_SIZE, IMAGE_SIZE, 3
    ):
        raise ValueError(
            f"Unexpected input shape: {recovered_model.input_shape}"
        )

    if recovered_model.output_shape[-1] != N_CLASSES:
        raise ValueError(
            "The saved model does not have seven expression outputs."
        )

    # A simple loading check-not an accuracy test.
    test_image = np.full(
        (1, IMAGE_SIZE, IMAGE_SIZE, 3),
        127.5,
        dtype=np.float32,
    )

    test_scores = np.asarray(
        recovered_model(test_image, training=False).numpy(),
        dtype=np.float32,
    )

    if test_scores.shape != (1, N_CLASSES):
        raise ValueError("The model returned an unexpected output shape.")

```

```

if not np.isfinite(test_scores).all():
    raise ValueError("The model returned invalid values.")

if not np.allclose(
    test_scores.sum(axis=1),
    1.0,
    atol=1e-3,
):
    raise ValueError("The model scores do not sum to one.")

# Restore the variable name used by the notebook.
model = recovered_model

FP32_PATH = OUTPUT_DIR / "expression_fp32.tflite"
INT8_PATH = OUTPUT_DIR / "expression_int8.tflite"

print("\nMODEL RESTORED: loading and test prediction passed.")
print("No training was run.")
print("This test checks loading, not model accuracy.")

# Read any results already saved before the crash.
print("\nSAVED EVALUATION REPORTS")

for filename in [
    "checkpoint_selection.csv",
    "holdout_results.csv",
    "format_validation.csv",
]:
    path = OUTPUT_DIR / filename

    if path.is_file():
        print(f"\n{filename}")
        display(pd.read_csv(path))
    else:
        print(f"\n{filename}: not found")

# Check whether the final Android bundle was already saved.
print("\nFINAL ANDROID FILES")

for filename in [
    "expression_android.tflite",
    "labels.txt",
    "model_settings.json",
]:
    path = OUTPUT_DIR / filename
    status = "FOUND" if path.is_file() else "NOT FOUND"
    print(f"{filename}: {status}")

Loading the saved model: /content/project_edge_ai/results/expression/20260915_092434/best_model.h5

MODEL RESTORED: loading and test prediction passed.
No training was run.
This test checks loading, not model accuracy.

SAVED EVALUATION REPORTS

checkpoint_selection.csv

  stage  validation_loss  validation_accuracy  validation_balanced_accuracy \
0  fine           1.011712           0.660256           0.470741
1  head           1.282925           0.567537           0.353249

  validation_recall_angry  validation_recall_disgust  validation_recall_fear \
0           0.487179           0.066845           0.160377
1           0.222222           0.013369           0.113208

  validation_recall_happy  validation_recall_sad  validation_recall_surprise \
0           0.716887           0.428844           0.648094
1           0.538022           0.274117           0.504399

  validation_recall_neutral
0           0.786963
1           0.807407

holdout_results.csv

```


	model	accuracy	balanced_accuracy	macro_f1
0	Majority-class baseline	0.381168	0.142857	0.078850
1	expression model	0.657742	0.460744	0.459526

format_validation.csv: not found

FINAL ANDROID FILES
 expression_android.tflite: NOT FOUND
 labels.txt: NOT FOUND
 model_settings.json: NOT FOUND

Code cell 46

```
# Check the saved mobile models and create the Android bundle.
# No training is run.

from pathlib import Path
from datetime import datetime, timezone
import hashlib
import json
import shutil

import numpy as np
import pandas as pd
import tensorflow as tf

from PIL import Image
from tqdm import tqdm
from IPython.display import display
from google.colab import files as colab_files

# -----
# 1. Find the saved run.
# -----
RUN_DIR = Path(
    "/content/project_edge_ai/results/expression/20260915_092434"
)

CLASS_NAMES = [
    "Angry", "Disgust", "Fear", "Happy",
    "Sad", "Surprise", "Neutral",
]

BOX_COLUMNS = [
    "face_x", "face_y", "face_width", "face_height",
]

N_CLASSES = 7
IMAGE_SIZE = 224
SEED = 42

# Keep the same limits declared in the original notebook.
LIMITS = {
    "FP32": {
        "max_abs": 0.02,
        "mean_abs": 0.002,
        "agreement": 0.99,
        "balanced_drop": 0.01,
        "recall_drop": 0.03,
    },
    "INT8": {
        "max_abs": 0.15,
        "mean_abs": 0.03,
        "agreement": 0.96,
        "balanced_drop": 0.03,
        "recall_drop": 0.05,
    },
}

for name in [
    "best_model.h5",
    "expression_fp32.tflite",
    "validation_images.csv",
```

```

]:
    if not (RUN_DIR / name).is_file():
        raise FileNotFoundError(
            f"Missing {name}. Keep the recovery ZIP. Do not retrain."
        )

# -----
# 2. Restore the saved validation images.
# -----
validation = pd.read_csv(
    RUN_DIR / "validation_images.csv"
)

needed = ["path", "label", *BOX_COLUMNS]

if not set(needed).issubset(validation.columns):
    raise ValueError(
        "The validation CSV is missing paths, labels or face boxes."
    )

# Reuse the original conversion sample if it survived.
sample_file = RUN_DIR / "conversion_validation_images.csv"

if sample_file.is_file():
    sample = pd.read_csv(sample_file)
else:
    count = min(
        64,
        int(validation["label"].value_counts().min()),
    )

    sample = pd.concat(
        [
            group.sample(
                count,
                random_state=SEED + int(label),
            )
            for label, group in validation.groupby("label")
        ],
        ignore_index=True,
    )

if not set(needed).issubset(sample.columns) or sample.empty:
    raise ValueError("The conversion sample is incomplete.")

# Every selected face must belong to the original validation set.
belongs_to_validation = pd.MultiIndex.from_frame(
    sample[needed]
).isin(
    pd.MultiIndex.from_frame(validation[needed])
)

if not belongs_to_validation.all():
    raise ValueError(
        "The conversion sample contains rows outside the saved validation set."
    )

if set(sample["label"].unique()) != set(range(N_CLASSES)):
    raise ValueError(
        "The validation sample must contain all seven expressions."
    )

sample = sample.reset_index(drop=True)
truth = sample["label"].to_numpy(dtype=np.int32)

print(
    "Validation images per class:",
    sample["label"].value_counts().sort_index().to_dict(),
)

missing = [
    path for path in sample["path"]

```

```

        if not Path(path).is_file()
    ]

if missing:
    raise FileNotFoundError(
        f"{len(missing)} validation images are missing. "
        f"Example: {missing[0]}. "
        "The trained model is safe. Restore the image dataset "
        "before checking the mobile exports."
    )

# -----
# 3. Use the original face preparation.
# -----
def read_face(row):

    with Image.open(row["path"]) as source:
        image = source.convert("RGB")

    x, y, width, height = [
        float(row[column])
        for column in BOX_COLUMNS
    ]

    if (x, y, width, height) != (-1, -1, -1, -1):
        if (
            not np.isfinite([x, y, width, height]).all()
            or min(width, height) <= 0
        ):
            raise ValueError("Invalid saved face box.")

        bounds = (
            max(0, int(x)),
            max(0, int(y)),
            min(image.width, int(x + width)),
            min(image.height, int(y + height)),
        )

        if bounds[2] <= bounds[0] or bounds[3] <= bounds[1]:
            raise ValueError("Face box is outside the image.")

        image = image.crop(bounds)

    if min(image.size) < 16:
        raise ValueError("Face crop is smaller than 16 pixels.")

    # Check against the saved image fingerprint when available.
    if "sha256" in row:
        digest = hashlib.sha256(
            str(image.size).encode() + image.tobytes()
        ).hexdigest()

        if digest != row["sha256"]:
            raise ValueError(
                "A validation face no longer matches its saved fingerprint."
            )

    image = image.resize(
        (IMAGE_SIZE, IMAGE_SIZE),
        Image.Resampling.BILINEAR,
    )

    return np.asarray(image, dtype=np.uint8)

pixels = np.stack(
    [
        read_face(row)
        for row in tqdm(
            sample.to_dict("records"),
            desc="Loading validation faces",
        )
    ]
)

```

```

# -----
# 4. Restore a float32 reference model on the CPU.
# -----
def clone_float32(layer):
    if isinstance(layer, tf.keras.Model):
        return tf.keras.models.clone_model(
            layer,
            clone_function=clone_float32,
        )

    config = layer.get_config()

    if "dtype" in config:
        config["dtype"] = "float32"

    return layer.__class__.from_config(config)

old_policy = tf.keras.mixed_precision.global_policy()

try:
    tf.keras.mixed_precision.set_global_policy("float32")

    with tf.device("/CPU:0"):
        saved = tf.keras.models.load_model(
            str(RUN_DIR / "best_model.h5"),
            compile=False,
        )

        reference = tf.keras.models.clone_model(
            saved,
            clone_function=clone_float32,
        )

        reference.set_weights(saved.get_weights())
        reference.trainable = False

finally:
    tf.keras.mixed_precision.set_global_policy(old_policy)

reference_parts = []

for start in tqdm(
    range(0, len(pixels), 8),
    desc="Checking saved Keras model on CPU",
):
    with tf.device("/CPU:0"):
        batch = pixels[start:start + 8].astype(np.float32)

        reference_parts.append(
            reference(batch, training=False).numpy()
        )

reference_scores = np.concatenate(reference_parts)

# -----
# 5. Read and run the existing TensorFlow Lite files.
# -----

def scale_and_zero(detail):
    parameters = detail["quantization_parameters"]
    scales = parameters["scales"]
    zeros = parameters["zero_points"]

    if (
        len(scales) != 1
        or len(zeros) != 1
        or not np.isfinite(scales[0])
        or scales[0] <= 0
    ):
        raise ValueError(
            "Expected valid per-tensor input/output quantization."

```

```

    )

    return float(scales[0]), int(zeros[0])

def lite_predict(path):
    engine = tf.lite.Interpreter(
        model_path=str(path),
        num_threads=2,
    )

    engine.allocate_tensors()

    inputs = engine.get_input_details()
    outputs = engine.get_output_details()

    if len(inputs) != 1 or len(outputs) != 1:
        raise ValueError("Expected one model input and one output.")

    inp = inputs[0]
    out = outputs[0]

    if (
        tuple(inp["shape"]) != (1, 224, 224, 3)
        or tuple(out["shape"]) != (1, 7)
    ):
        raise ValueError("Unexpected mobile model shape.")

    predictions = []

    for image in tqdm(pixels, desc="Checking " + path.name):
        value = image[None].astype(np.float32)

        if np.issubdtype(inp["dtype"], np.integer):
            scale, zero = scale_and_zero(inp)
            bounds = np.iinfo(inp["dtype"])

            value = np.clip(
                np rint(value / scale + zero),
                bounds.min,
                bounds.max,
            )

            engine.set_tensor(
                inp["index"],
                value.astype(inp["dtype"]),
            )

            engine.invoke()

            value = engine.get_tensor(
                out["index"]
            )[0].astype(np.float32)

            if np.issubdtype(out["dtype"], np.integer):
                scale, zero = scale_and_zero(out)
                value = (value - zero) * scale

            predictions.append(value)

    return np.asarray(predictions), inp, out

def validate_scores(scores, tolerance):
    if (
        scores.shape != (len(truth), N_CLASSES)
        or not np.isfinite(scores).all()
    ):
        raise ValueError(
            "Invalid output shape or non-finite model scores."
        )

    if (
        (scores < -tolerance).any()

```

```

        or (scores > 1 + tolerance).any()
    ):
        raise ValueError(
            "Model output is outside the probability range."
        )

    if not np.allclose(
        scores.sum(axis=1),
        1,
        atol=tolerance,
        rtol=0,
    ):
        raise ValueError(
            "Model probabilities do not sum to one."
        )

def check_conversion(base, candidate, name):
    validate_scores(base, 1e-4)

    validate_scores(
        candidate,
        0.03 if name == "INT8" else 1e-4,
    )

    base_labels = base.argmax(axis=1)
    candidate_labels = candidate.argmax(axis=1)

    base_recall = np.array([
        (base_labels[truth == k] == k).mean()
        for k in range(N_CLASSES)
    ])

    candidate_recall = np.array([
        (candidate_labels[truth == k] == k).mean()
        for k in range(N_CLASSES)
    ])

    difference = np.abs(base - candidate)
    limit = LIMITS[name]

    result = {
        "format": name,
        "max_abs_difference": float(difference.max()),
        "mean_abs_difference": float(difference.mean()),
        "prediction_agreement": float(
            (base_labels == candidate_labels).mean()
        ),
        "accuracy": float(
            (candidate_labels == truth).mean()
        ),
        "balanced_accuracy": float(candidate_recall.mean()),
        "balanced_accuracy_drop": float(
            base_recall.mean() - candidate_recall.mean()
        ),
        "largest_class_recall_drop": float(
            (base_recall - candidate_recall).max()
        ),
    }

    failures = []

    for key, maximum in [
        ("max_abs_difference", limit["max_abs"]),
        ("mean_abs_difference", limit["mean_abs"]),
        ("balanced_accuracy_drop", limit["balanced_drop"]),
        ("largest_class_recall_drop", limit["recall_drop"]),
    ]:
        if result[key] > maximum + 1e-12:
            failures.append(key)

    if (
        result["prediction_agreement"] + 1e-12

```

```

        < limit["agreement"]
    ):
        failures.append("prediction_agreement")

    result.update({
        f"recall_{name.lower()}": float(recall)
        for name, recall in zip(CLASS_NAMES, candidate_recall)
    })

    result["passed"] = not failures
    result["reason"] = (
        ", ".join(failures) or "All checks passed"
    )

    return result

# -----
# 6. Validate FP32, then try INT8.
# -----

stamp = datetime.now(timezone.utc).strftime(
    "%Y%m%d_%H%M%S%f"
)

# Save separately so the original run is not overwritten.
BUNDLE_DIR = RUN_DIR / ("android_recovered_" + stamp)
BUNDLE_DIR.mkdir(parents=True, exist_ok=False)

sample.to_csv(
    BUNDLE_DIR / "conversion_validation_images.csv",
    index=False,
)

(BUNDLE_DIR / "conversion_limits.json").write_text(
    json.dumps(LIMITS, indent=2),
    encoding="utf-8",
)

fp32_path = RUN_DIR / "expression_fp32.tflite"

fp32_scores, chosen_input, chosen_output = lite_predict(
    fp32_path
)

fp32_report = check_conversion(
    reference_scores,
    fp32_scores,
    "FP32",
)

reports = [fp32_report]

pd.DataFrame(reports).to_csv(
    BUNDLE_DIR / "format_validation.csv",
    index=False,
)

if not fp32_report["passed"]:
    display(pd.DataFrame(reports))

    raise RuntimeError(
        "FP32 failed the conversion checks. "
        "No Android model was approved. "
        "The trained model and original files are unchanged."
    )

chosen_format = "FP32"
chosen_path = fp32_path

int8_path = RUN_DIR / "expression_int8.tflite"

if int8_path.is_file():
    try:
        int8_scores, int8_input, int8_output = lite_predict(

```

```

        int8_path
    )

    int8_report = check_conversion(
        fp32_scores,
        int8_scores,

        "INT8",
    )

    reports.append(int8_report)

    if (
        int8_report["passed"]
        and int8_path.stat().st_size < fp32_path.stat().st_size
    ):
        chosen_format = "INT8"
        chosen_path = int8_path
        chosen_input = int8_input
        chosen_output = int8_output

except Exception as error:
    reports.append({
        "format": "INT8",
        "passed": False,
        "reason": str(error),
    })

    print(
        "INT8 was not accepted. Keeping the validated FP32 model."
    )

report = pd.DataFrame(reports)

report.to_csv(
    BUNDLE_DIR / "format_validation.csv",
    index=False,
)

display(report[
    [
        "format",
        "passed",
        "prediction_agreement",
        "balanced_accuracy",
        "reason",
    ]
])

# -----
# 7. Save the three Android files and supporting reports.
# -----
def tensor_rules(detail):
    if np.issubdtype(detail["dtype"], np.integer):
        scale, zero = scale_and_zero(detail)
    else:
        scale, zero = 0.0, 0

    return {
        "shape": detail["shape"].tolist(),
        "dtype": np.dtype(detail["dtype"]).name,
        "scale": scale,
        "zero_point": zero,
    }

settings = {
    "course": "DLBAIPEAI",
    "task": "Task 2",
    "model_task": "expression",
    "architecture": "EfficientNetV2B0",
    "dataset": "Expw",
    "class_names": CLASS_NAMES,
    "input": tensor_rules(chosen_input),

```



```

"output": tensor_rules(chosen_output),
"chosen_format": chosen_format,
"source_run": str(RUN_DIR),
"image_rule": (
    "Crop face, convert to RGB, resize to 224x224 "
    "using bilinear interpolation."
),
"pixel_rule": (
    "Float pixels 0-255. Scaling is inside the model. "
    "Do not divide by 255."
),
"integer_rule": (
    "Use the saved tensor scale and zero point, "
    "with rounding and clipping."
),
"includes_face_detector": False,
"measured_on_phone": False,
"training_rerun": False,
"conversion_validation_passed": True,
"conversion_validation_images": len(sample),
"conversion_limits": LIMITS,
"tensorflow_recovery_version": tf.__version__,

"training_seconds": None,
"training_time_note": (
    "Original timing variable was lost. No time is invented."
),
}

for filename in [
    "checkpoint_selection.csv",
    "holdout_results.csv",
    "classification_report.csv",
    "happy_sad_results.csv",
]:
    if (RUN_DIR / filename).is_file():
        shutil.copy2(
            RUN_DIR / filename,
            BUNDLE_DIR / filename,
        )

# Preserve the original Keras holdout results without relabelling
# them as new TensorFlow Lite or phone results.
if (RUN_DIR / "holdout_results.csv").is_file():
    old_results = pd.read_csv(
        RUN_DIR / "holdout_results.csv"
    )

    rows = old_results.loc[
        old_results["model"].eq("expression model")
    ]

    if len(rows) == 1:
        settings["holdout_scores"] = {
            key: float(rows.iloc[0][key])
            for key in [
                "accuracy",
                "balanced_accuracy",
                "macro_f1",
            ]
        }

        settings["holdout_scores_scope"] = (
            "Original saved Keras results, not a new mobile holdout test."
        )

android_path = BUNDLE_DIR / "expression_android.tflite"

shutil.copy2(chosen_path, android_path)

settings["model_sha256"] = hashlib.sha256(
    android_path.read_bytes()
).hexdigest()

```

```

(BUNDLE_DIR / "labels.txt").write_text(
    "\n".join(CLASS_NAMES) + "\n",
    encoding="utf-8",
)

(BUNDLE_DIR / "model_settings.json").write_text(
    json.dumps(settings, indent=2, allow_nan=False),
    encoding="utf-8",
)

archive = shutil.make_archive(
    str(RUN_DIR / ("expression_android_bundle_" + stamp)),
    "zip",
    root_dir=BUNDLE_DIR,
)

print("\nSelected model:", chosen_format)
print("Android files saved:", BUNDLE_DIR)
print("No training or new dataset download was run.")

colab_files.download(archive)

Validation images per class: {0: 64, 1: 64, 2: 64, 3: 64, 4: 64, 5: 64, 6: 64}

Loading validation faces: 100%|#####| 448/448 [00:09<00:00, 49.71it/s]
Checking saved Keras model on CPU: 100%|#####| 56/56 [00:48<00:00, 1.15it/s]
/usr/local/lib/python3.13/dist-packages/tensorflow/lite/python/interpreter.py:457: UserWarning:
Warning: tf.lite.Interpreter is deprecated and is scheduled for deletion in
TF 2.20. Please use the LiteRT interpreter from the ai_edge_litert package.
See the [migration guide](https://ai.google.dev/edge/litert/migration)
for details.

warnings.warn(_INTERPRETER_DELETION_WARNING)
Checking expression_fp32.tflite: 100%|#####| 448/448 [00:12<00:00, 36.96it/s]
Checking expression_int8.tflite: 100%|#####| 448/448 [00:12<00:00, 35.25it/s]

format passed prediction_agreement balanced_accuracy \
0 FP32 True 1.000000 0.468750
1 INT8 False 0.857143 0.441964

reason
0 All checks passed
1 max_abs_difference, largest_class_recall_drop,...

Selected model: FP32
Android files saved:
/content/project_edge_ai/results/expression/20260915_092434/android_recovered_20260915_142717_617660
No training or new dataset download was run.

<IPython.core.display.Javascript object>

<IPython.core.display.Javascript object>

```

22. Keep the best model

In this cell we compare the two saved checkpoints. Validation balanced accuracy chooses the model, and validation loss only breaks a tie. The holdout does not choose the winner.

Code cell 48

```

# cell:select
OUTPUT_DIR.mkdir(
    parents=True,
    exist_ok=True,
)

choices = []

for stage in ["head", "fine"]:
    model.load_weights(
        str(
            OUTPUT_DIR
            / f"best_{stage}.weights.h5"
        )
    )

```

```

compile_model(
    model,
    1e-5,
)

result = model.evaluate(
    val_ds,
    verbose=0,
    return_dict=True,
)

row = {
    "stage": stage,
    "validation_loss": float(result["loss"]),

    "validation_accuracy": float(result["accuracy"]),
    "validation_balanced_accuracy": float(
        result["balanced_accuracy"]
    ),
}

for class_name, metric_name in zip(
    CLASS_NAMES,
    RECALL_NAMES,
):
    row[
        "validation_" + metric_name
    ] = float(result[metric_name])

choices.append(row)

selection = (
    pd.DataFrame(choices)
    .sort_values(
        [
            "validation_balanced_accuracy",
            "validation_loss",
        ],
        ascending=[False, True],
    )
    .reset_index(drop=True)
)

display(selection)

selection.to_csv(
    OUTPUT_DIR / "checkpoint_selection.csv",
    index=False,
)

BEST_STAGE = str(
    selection.iloc[0]["stage"]
)

model.load_weights(
    str(
        OUTPUT_DIR
        / f"best_{BEST_STAGE}.weights.h5"
    )
)

model.save(
    OUTPUT_DIR / "best_model.h5",
    include_optimizer=False,
)

print("Chosen stage:", BEST_STAGE)

-----
NameError                                Traceback (most recent call last)
/tmp/ipykernel_76412/305169044.py in <cell line: 0>()
      1 # cell:select
----> 2 OUTPUT_DIR.mkdir(
      3     parents=True,

```

```

4     exist_ok=True,
5 )

```

NameError: name 'OUTPUT_DIR' is not defined

23. Plot the training results

In this cell we plot loss, accuracy, balanced accuracy and the validation recall for every expression. The dashed line shows when fine-tuning started.

Code cell 50

```

# cell:curves
OUTPUT_DIR.mkdir(
    parents=True,
    exist_ok=True,
)

history = pd.concat(
    [
        pd.read_csv(
            OUTPUT_DIR / "head_training.csv"
        ),
        pd.read_csv(
            OUTPUT_DIR / "fine_training.csv"
        ),
    ],
    ignore_index=True,
)

history.to_csv(
    OUTPUT_DIR / "training_history.csv",
    index=False,
)

fine_start = (
    len(head_history.history["loss"])
    + 0.5
)

for metric in [
    "loss",

    "accuracy",
    "balanced_accuracy",
]:
    fig, ax = plt.subplots(
        figsize=(8, 4)
    )

    ax.plot(
        np.arange(1, len(history) + 1),
        history[metric],
        label="Training",
    )

    ax.plot(
        np.arange(1, len(history) + 1),
        history["val_" + metric],
        label="Validation",
    )

    ax.axvline(
        fine_start,
        linestyle="--",
    )

    ax.set(
        xlabel="Epoch",
        ylabel=metric.replace("_", " ").title(),
        title=metric.replace("_", " ").title()
        + " during training",
    )

```

```

ax.legend()

fig.tight_layout()

fig.savefig(
    OUTPUT_DIR
    / f"training_{metric}.png",
    dpi=150,
)

plt.show()

# Show validation recall for all seven expressions on one graph.
fig, ax = plt.subplots(
    figsize=(10, 5)
)

for class_name, metric_name in zip(
    CLASS_NAMES,
    RECALL_NAMES,
):
    ax.plot(
        np.arange(1, len(history) + 1),
        history["val_" + metric_name],
        label=class_name,
    )

ax.axvline(
    fine_start,
    linestyle="--",
)

ax.set(
    xlabel="Epoch",
    ylabel="Validation recall",
    title="Validation recall for each expression",
)

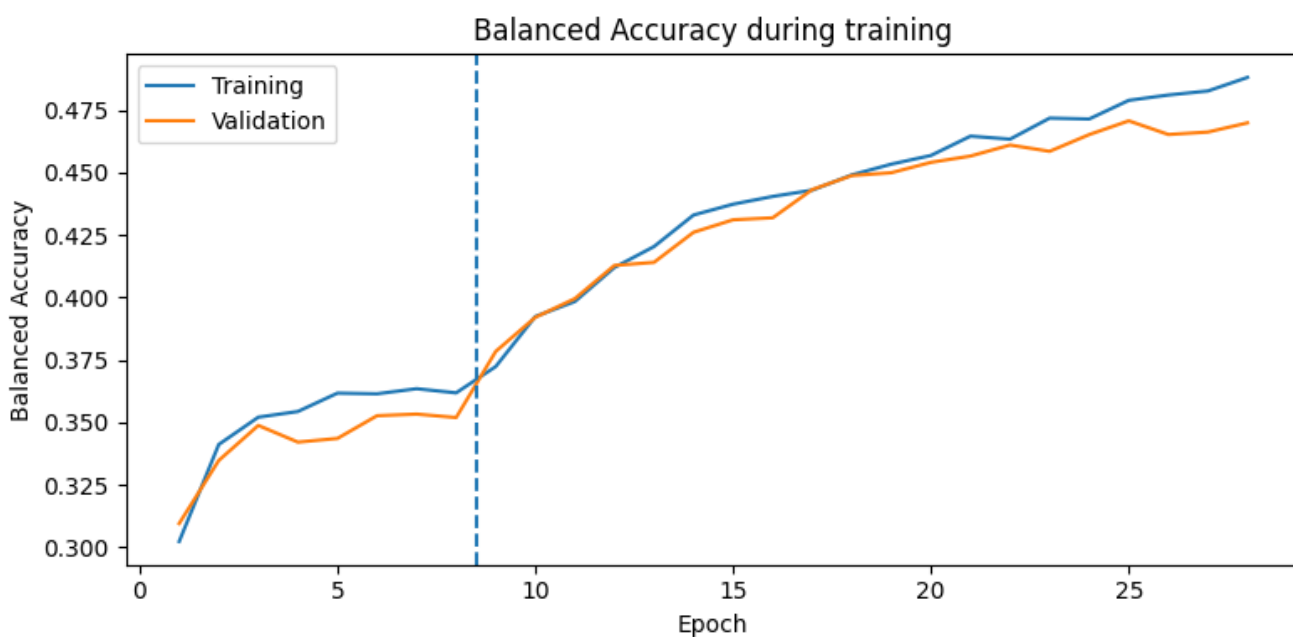
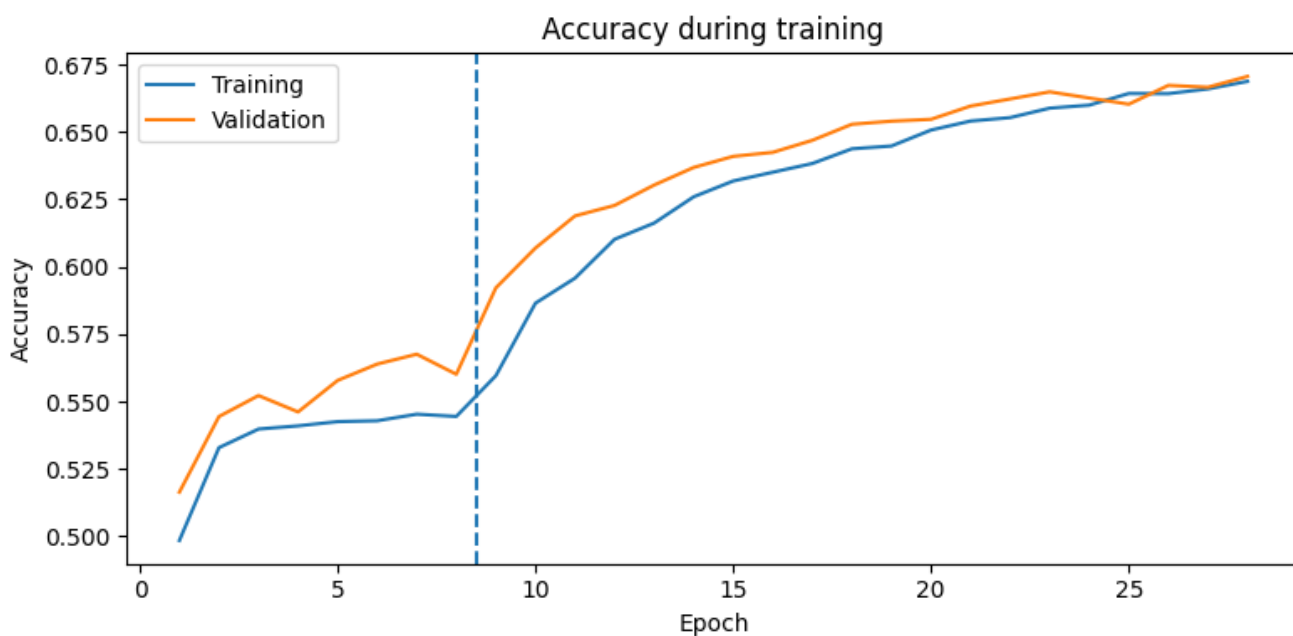
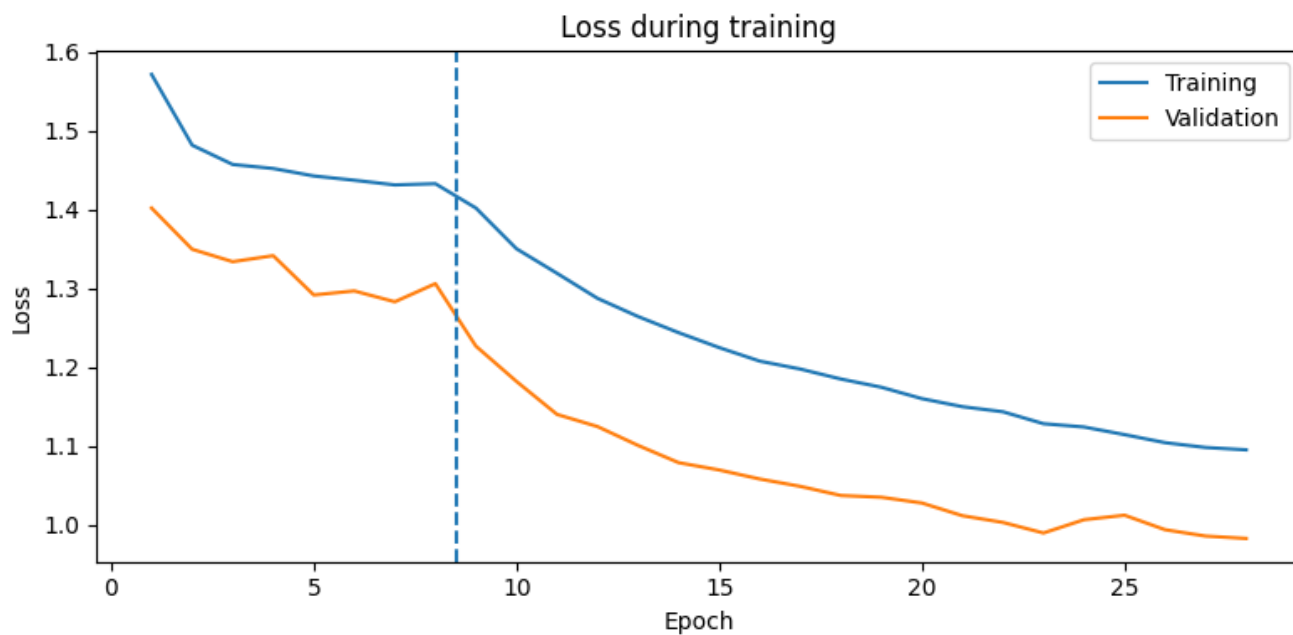
ax.legend(
    ncol=2,
)

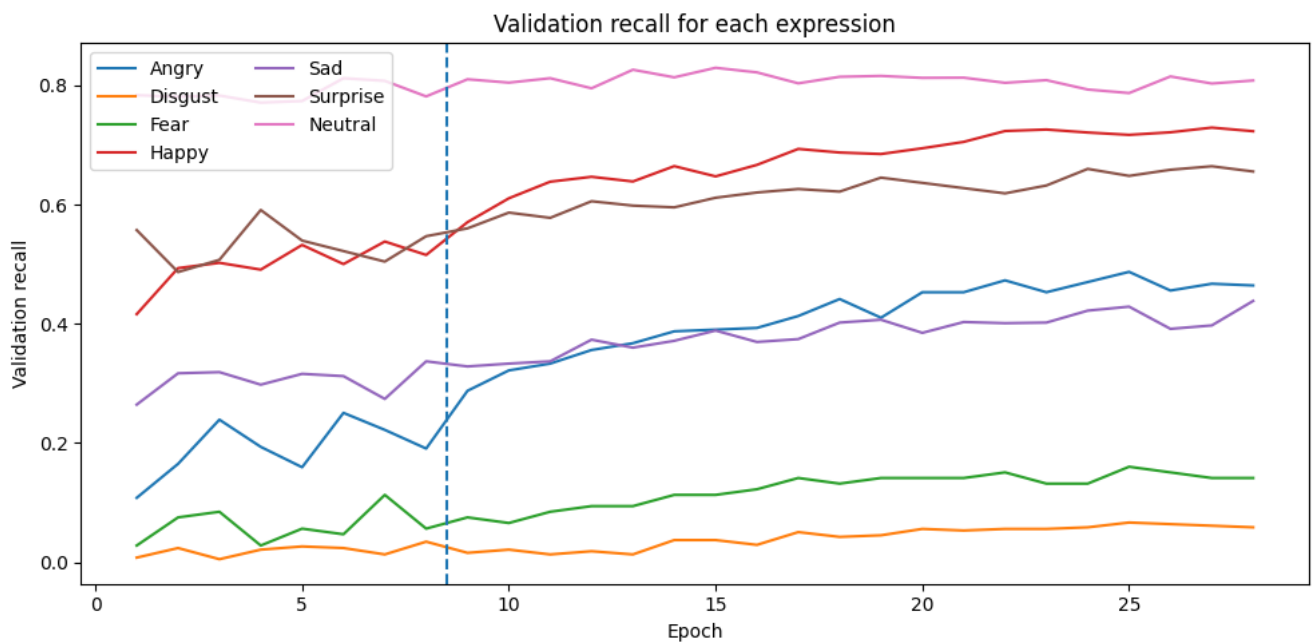
fig.tight_layout()

fig.savefig(
    OUTPUT_DIR
    / "validation_recall_by_expression.png",
    dpi=150,
)

plt.show()

```





24. Test the model on the holdout

In this cell we test the chosen model on data that did not guide training. We compare it with a simple baseline that always predicts the largest training class. Macro F1 gives each class equal importance.

Code cell 52

```
# cell:evaluate
OUTPUT_DIR.mkdir(parents=True, exist_ok=True)
probabilities = model.predict(test_ds, verbose=0)
if not np.isfinite(probabilities).all():
    raise ValueError("The model produced invalid scores.")
y_true = test_df["label"].to_numpy()
y_pred = probabilities.argmax(axis=1)
majority_class = int(train_df["label"].value_counts().idxmax())
baseline_pred = np.full(len(y_true), majority_class)

def scores(truth, prediction):
    return {"accuracy": float(accuracy_score(truth, prediction)),
            "balanced_accuracy": float(balanced_accuracy_score(truth, prediction)),
            "macro_f1": float(f1_score(truth, prediction, average="macro",
                                         labels=list(range(N_CLASSES)), zero_division=0))}

model_scores = scores(y_true, y_pred)
results = pd.DataFrame([{"model": "Majority-class baseline", **scores(y_true, baseline_pred)},
                        {"model": TASK + " model", **model_scores}])
display(results)
results.to_csv(OUTPUT_DIR / "holdout_results.csv", index=False)
predictions = test_df.copy()
predictions["true_class"] = [CLASS_NAMES[value] for value in y_true]
predictions["predicted_class"] = [CLASS_NAMES[value] for value in y_pred]
predictions["model_score"] = probabilities.max(axis=1)
predictions["correct"] = y_true == y_pred
predictions.to_csv(OUTPUT_DIR / "holdout_predictions.csv", index=False)
```

	model	accuracy	balanced_accuracy	macro_f1
0	Majority-class baseline	0.381168	0.142857	0.078850
1	expression model	0.657742	0.460744	0.459526

25. Check each class

In this cell we show results for every class. The confusion matrix shows the real labels in rows and predictions in columns.

Code cell 54

```
# cell:class_report
OUTPUT_DIR.mkdir(parents=True, exist_ok=True)
report = pd.DataFrame(classification_report(y_true, y_pred, labels=list(range(N_CLASSES)),
                                          target_names=CLASS_NAMES, output_dict=True, zero_division=0)).T

display(report)
report.to_csv(OUTPUT_DIR / "classification_report.csv")
cm = confusion_matrix(y_true, y_pred, labels=list(range(N_CLASSES)))
pd.DataFrame(cm, index=CLASS_NAMES, columns=CLASS_NAMES).to_csv(OUTPUT_DIR / "confusion_matrix.csv")
fig, ax = plt.subplots(figsize=(8, 7))
ConfusionMatrixDisplay(cm, display_labels=CLASS_NAMES).plot(ax=ax, colorbar=False,
                                                            xticks_rotation=45)

fig.tight_layout()
fig.savefig(OUTPUT_DIR / "confusion_matrix.png", dpi=150)
plt.show()
```

```
-----
NameError                                Traceback (most recent call last)
/tmp/ipykernel_76412/3898518992.py in <cell line: 0>()
      1 # cell:class_report
----> 2 OUTPUT_DIR.mkdir(parents=True, exist_ok=True)
      3 report = pd.DataFrame(classification_report(y_true, y_pred, labels=list(range(N_CLASSES)),
      4                                          target_names=CLASS_NAMES, output_dict=True, zero_division=0)).T
      5 display(report)
```

NameError: name 'OUTPUT_DIR' is not defined

26. Check happy and sad faces

In this cell we check the two expressions named in the course experiment. The model still chooses from all seven classes. We do not make the test easier by forcing a happy-or-sad answer.

Code cell 56

```
# cell:task_check
OUTPUT_DIR.mkdir(
    parents=True,
    exist_ok=True,
)

HAPPY_LABEL = CLASS_NAMES.index("Happy")
SAD_LABEL = CLASS_NAMES.index("Sad")

if (HAPPY_LABEL, SAD_LABEL) != (3, 4):
    raise RuntimeError(
        "The Happy/Sad class order changed. "
        "Check CLASS_NAMES before evaluating the course task."
    )

happy_sad = predictions.loc[
    predictions["label"].isin(
        [
            HAPPY_LABEL,
            SAD_LABEL,
        ]
    )
].copy()

happy_sad_results = (
    happy_sad
    .groupby("true_class")
    .agg(
        images=("correct", "size"),
        recall=("correct", "mean"),
    )
)

display(happy_sad_results)

happy_sad_results.to_csv(
    OUTPUT_DIR
    / "happy_sad_results.csv"
```



```

)

print(
    "Happy and Sad are evaluated as part of the full "
    "seven-class model. Predictions are not forced to only two choices."
)

-----
NameError                                Traceback (most recent call last)
/tmp/ipykernel_76412/2959382591.py in <cell line: 0>()
      1 # cell:task_check
----> 2 OUTPUT_DIR.mkdir(
      3     parents=True,
      4     exist_ok=True,
      5 )

NameError: name 'OUTPUT_DIR' is not defined

```

27. Look at the mistakes

In this cell we show mistakes with high model scores. A high score is not a guarantee that the prediction is correct.

Code cell 58

```

# cell:errors
OUTPUT_DIR.mkdir(parents=True, exist_ok=True)
mistakes = predictions.loc[~predictions["correct"]].sort_values("model_score", ascending=False)
display(mistakes[["true_class", "predicted_class", "model_score"]].head(10))
mistakes.to_csv(OUTPUT_DIR / "mistakes.csv", index=False)
if SHOW_FACE_IMAGES:
    for _, row in mistakes.head(3).iterrows():
        fig, ax = plt.subplots(figsize=(3, 3))
        ax.imshow(open_face(row["path"], row[BOX_COLUMNS]))
        ax.set_title(f"Label: {row['true_class']}\nPrediction: {row['predicted_class']}")
        ax.axis("off")
        plt.show()

-----
NameError                                Traceback (most recent call last)
/tmp/ipykernel_76412/2383617790.py in <cell line: 0>()
      1 # cell:errors
----> 2 OUTPUT_DIR.mkdir(parents=True, exist_ok=True)
      3 mistakes = predictions.loc[~predictions["correct"]].sort_values("model_score",
ascending=False)
      4 display(mistakes[["true_class", "predicted_class", "model_score"]].head(10))
      5 mistakes.to_csv(OUTPUT_DIR / "mistakes.csv", index=False)

NameError: name 'OUTPUT_DIR' is not defined

```

28. Export the normal Android model

In this cell we rebuild the chosen network in float32 on the CPU and convert it to TensorFlow Lite. This gives the conversion check a clean reference that does not depend on GPU mixed precision.

Code cell 60

```

# cell:fp32
OUTPUT_DIR.mkdir(
    parents=True,
    exist_ok=True,
)

# Training uses mixed_float16 on the GPU.
# For deployment checking we build the same architecture in float32 on CPU.
training_policy = tf.keras.mixed_precision.global_policy()

tf.keras.mixed_precision.set_global_policy(
    "float32"
)

try:
    with tf.device("/CPU:0"):

```

```

cpu_reference, _ = build_expression_model(
    weights=None
)

cpu_reference.set_weights(
    model.get_weights()
)

cpu_reference.trainable = False

cpu_reference(
    tf.zeros(
        (
            1,
            IMAGE_SIZE,
            IMAGE_SIZE,
            3,
        ),
        dtype=tf.float32,
    ),
    training=False,
)

converter = tf.lite.TFLiteConverter.from_keras_model(
    cpu_reference
)

converter.target_spec.supported_ops = [
    tf.lite.OpsSet.TFLITE_BUILTINS
]

fp32_bytes = converter.convert()

finally:
    tf.keras.mixed_precision.set_global_policy(
        training_policy
    )

FP32_PATH = (
    OUTPUT_DIR
    / f"{TASK}_fp32.tflite"
)

FP32_PATH.write_bytes(
    fp32_bytes
)

print("Saved:", FP32_PATH)
print(
    "Size in MB:",
    round(
        FP32_PATH.stat().st_size / 1e6,
        2,
    ),
)

-----
NameError                                Traceback (most recent call last)
/tmp/ipykernel_76412/3225513862.py in <cell line: 0>()
      1 # cell:fp32
----> 2 OUTPUT_DIR.mkdir(
      3     parents=True,
      4     exist_ok=True,
      5 )

NameError: name 'OUTPUT_DIR' is not defined

```

29. Try a smaller Android model

In this cell we try INT8 conversion. It uses smaller numbers and may change accuracy. Only training images set the number ranges. If conversion fails, we keep the normal model and record the error.

Code cell 62

```
# cell:int8
OUTPUT_DIR.mkdir(parents=True, exist_ok=True)
calibration_df = pd.concat([group.sample(min(40, len(group)), random_state=SEED)
                           for _, group in train_df.groupby("label")])
calibration_df.to_csv(OUTPUT_DIR / "calibration_images.csv", index=False)

def representative_images():
    for images, _ in make_dataset(calibration_df):
        for image in images.numpy():
            yield [image[None].astype(np.float32)]

INT8_PATH = None
try:
    converter = tf.lite.TFLiteConverter.from_keras_model(cpu_reference)
    converter.optimizations = [tf.lite.Optimize.DEFAULT]
    converter.representative_dataset = representative_images
    converter.target_spec.supported_ops = [tf.lite.OpsSet.TFLITE_BUILTINS_INT8]
    converter.inference_input_type = tf.int8
    converter.inference_output_type = tf.int8
    converted = converter.convert()
    INT8_PATH = OUTPUT_DIR / f"{TASK}_int8.tflite"
    INT8_PATH.write_bytes(converted)
    print("Saved:", INT8_PATH)
except Exception as error:
    print("INT8 conversion failed. The FP32 model is still available.")
    print(type(error).__name__ + ":", error)
    (OUTPUT_DIR / "int8_error.txt").write_text(str(error), encoding="utf-8")

-----
NameError                                Traceback (most recent call last)
/tmp/ipykernel_76412/3694535794.py in <cell line: 0>()
      1 # cell:int8
----> 2 OUTPUT_DIR.mkdir(parents=True, exist_ok=True)
      3 calibration_df = pd.concat([group.sample(min(40, len(group)), random_state=SEED)
      4                               for _, group in train_df.groupby("label")])
      5 calibration_df.to_csv(OUTPUT_DIR / "calibration_images.csv", index=False)

NameError: name 'OUTPUT_DIR' is not defined
```

30. Check the converted models

In this cell we compare TensorFlow Lite with a separate float32 Keras model on the CPU. The check uses the same number of images from every expression class and compares prediction agreement, balanced accuracy and class recall as well as score differences.

Code cell 64

```
# cell:mobile_check
OUTPUT_DIR.mkdir(
    parents=True,
    exist_ok=True,
)

def balanced_check_sample(
    frame,
    per_class,
    seed,
):
    parts = []

    for label, group in frame.groupby(
        "label"
    ):
        count = min(
            per_class,
            len(group),
        )

        parts.append(
            group.sample(
```

```

        count,
        random_state=seed + int(label),
    )
)

result = pd.concat(
    parts,
    ignore_index=True,
)

if set(result["label"]) != set(range(N_CLASSES)):
    raise ValueError(
        "The conversion sample must contain all seven expressions."
    )

return result


def probability_check(
    values,
    rows,
    tolerance=1e-4,
):
    values = np.asarray(
        values,
        dtype=np.float32,
    )

    if (
        values.shape
        != (rows, N_CLASSES)
        or not np.isfinite(values).all()
    ):
        raise ValueError(
            "Wrong prediction shape or invalid model scores."
        )

    if (
        (values < -tolerance).any()
        or (values > 1 + tolerance).any()
    ):
        raise ValueError(
            "Model scores are outside the probability range."
        )

    if not np.allclose(
        values.sum(axis=1),
        1,
        rtol=0,
        atol=tolerance,
    ):
        raise ValueError(
            "Class probabilities do not sum to one."
        )

    return values


def class_recalls(
    truth,
    prediction,
):
    truth = np.asarray(truth)
    prediction = np.asarray(prediction)

    if set(np.unique(truth)) != set(range(N_CLASSES)):
        raise ValueError(
            "The check sample must contain all seven expression classes."
        )

    return np.array(
        [
            np.mean(

```

```

        prediction[truth == class_id]
        == class_id
    )
    for class_id in range(N_CLASSES)
],
dtype=np.float32,
)

def conversion_report(
    truth,
    reference,
    candidate,
    limits,
    quantized=False,
):
    truth = np.asarray(truth)

    reference = probability_check(
        reference,
        len(truth),
    )

    candidate = probability_check(
        candidate,
        len(truth),
        tolerance=0.03 if quantized else 1e-4,
    )

    difference = np.abs(
        reference - candidate
    )

    reference_labels = reference.argmax(
        axis=1
    )

    candidate_labels = candidate.argmax(
        axis=1
    )

    reference_recall = class_recalls(
        truth,
        reference_labels,
    )

    candidate_recall = class_recalls(
        truth,
        candidate_labels,
    )

    result = {
        "max_abs_difference": float(
            difference.max()
        ),
        "mean_abs_difference": float(
            difference.mean()
        ),
        "p99_abs_difference": float(
            np.quantile(
                difference,
                0.99,
            )
        ),
        "prediction_agreement": float(
            np.mean(
                reference_labels
                == candidate_labels
            )
        ),
        "accuracy": float(
            np.mean(
                truth
                == candidate_labels
            )
        )
    }

```

```

        )
    ),
    "balanced_accuracy": float(
        candidate_recall.mean()
    ),
    "balanced_accuracy_drop": float(
        reference_recall.mean()
        - candidate_recall.mean()
    ),
    "largest_class_recall_drop": float(
        np.max(
            reference_recall
            - candidate_recall
        )
    ),
}

for class_name, value in zip(
    CLASS_NAMES,
    candidate_recall,
):
    result[
        "recall_"
        + class_name.lower()
    ] = float(value)

reasons = []

checks = [
    (
        "max_abs_difference",
        "max_abs",
    ),
    (
        "mean_abs_difference",
        "mean_abs",
    ),
    (
        "balanced_accuracy_drop",
        "max_balanced_accuracy_drop",
    ),
    (
        "largest_class_recall_drop",
        "max_class_recall_drop",
    ),
]

for key, limit_key in checks:
    if (
        result[key]
        > limits[limit_key]
        + 1e-12
    ):
        reasons.append(
            key
            + " exceeds the set limit"
        )

if (
    result["prediction_agreement"]
    + 1e-12
    < limits["min_agreement"]
):
    reasons.append(
        "prediction agreement is below the set limit"
    )

result["passed"] = not reasons
result["reason"] = (
    "; ".join(reasons)
    or "All checks passed"
)

```

```

    return result

def encode_input(
    value,
    detail,
):
    value = np.asarray(
        value,
        dtype=np.float32,
    )

    if not np.isfinite(value).all():
        raise ValueError(
            "Input contains invalid pixels."
        )

    if np.issubdtype(
        detail["dtype"],
        np.integer,
    ):
        scale, zero = detail[
            "quantization"
        ]

        if (
            not np.isfinite(scale)
            or scale <= 0
        ):
            raise ValueError(
                "Invalid input quantization scale."
            )

        limits = np.iinfo(
            detail["dtype"]
        )

        value = np.clip(
            np rint(
                value / scale
                + zero
            ),
            limits.min,
            limits.max,
        )

    return value.astype(
        detail["dtype"]
    )

def decode_output(
    value,
    detail,
):
    value = np.asarray(value)

    if np.issubdtype(
        detail["dtype"],
        np.integer,
    ):
        scale, zero = detail[
            "quantization"
        ]

        if (
            not np.isfinite(scale)
            or scale <= 0
        ):
            raise ValueError(
                "Invalid output quantization scale."
            )

    value = (

```

```

        value.astype(np.float32)
        - zero
    ) * scale

    return value.astype(
        np.float32
    )

def open_lite(path):
    interpreter = tf.lite.Interpreter(
        model_path=str(path),

        num_threads=4,
    )

    interpreter.allocate_tensors()

    inputs = interpreter.get_input_details()
    outputs = interpreter.get_output_details()

    if len(inputs) != 1 or len(outputs) != 1:
        raise ValueError(
            "Expected one image input and one expression output."
        )

    expected_input = [
        1,
        IMAGE_SIZE,
        IMAGE_SIZE,
        3,
    ]

    expected_output = [
        1,
        N_CLASSES,
    ]

    if (
        list(inputs[0]["shape"])
        != expected_input
        or list(outputs[0]["shape"])
        != expected_output
    ):
        raise ValueError(
            "Unexpected TensorFlow Lite input/output shape."
        )

    return (
        interpreter,
        inputs[0],
        outputs[0],
    )

def lite_predictions(
    path,
    frame,
):
    interpreter, input_info, output_info = open_lite(
        path
    )

    values = []

    for images, _ in make_dataset(frame):
        for image in images.numpy():
            interpreter.set_tensor(
                input_info["index"],
                encode_input(
                    image[None],
                    input_info,
                ),
            ),

```



```

        )

        interpreter.invoke()

        values.append(
            decode_output(
                interpreter.get_tensor(
                    output_info["index"]
                )[0],
                output_info,
            )
        )

    return np.asarray(
        values,
        dtype=np.float32,
    )

def cpu_predictions(frame):
    values = []

    for images, _ in make_dataset(frame):
        with tf.device("/CPU:0"):
            values.append(
                cpu_reference(
                    images,
                    training=False,
                ).numpy()
            )

    return np.concatenate(
        values
    )

validation_sample = balanced_check_sample(
    val_df,
    MOBILE_IMAGES_PER_CLASS,
    SEED,
)

validation_sample.to_csv(
    OUTPUT_DIR
    / "conversion_validation_images.csv",
    index=False,
)

truth = validation_sample[
    "label"
].to_numpy()

keras_scores = cpu_predictions(
    validation_sample
)

fp32_scores = lite_predictions(
    FP32_PATH,
    validation_sample,
)

fp32_report = conversion_report(
    truth,
    keras_scores,
    fp32_scores,
    FP32_LIMITS,
)

# GPU/CPU drift is diagnostic only.
gpu_scores = model.predict(
    make_dataset(validation_sample),
    verbose=0,
)

print(

```

```

    "Largest GPU-Keras / CPU-Keras difference:",
    float(
        np.max(
            np.abs(
                gpu_scores
                - keras_scores
            )
        )
    ),
)

print(
    "Largest CPU-Keras / FP32-Lite difference:",
    fp32_report["max_abs_difference"],
)

print(
    "FP32 class-prediction agreement:",
    f"{fp32_report['prediction_agreement']:.2%}",
)

format_rows = [
    dict(
        format="FP32",
        size_MB=(
            FP32_PATH.stat().st_size
            / 1e6
        ),
        **fp32_report,
    )
]

mobile_models = {
    "FP32": FP32_PATH
}

CHOSEN_FORMAT = "FP32"

if INT8_PATH is not None:
    try:
        int8_scores = lite_predictions(
            INT8_PATH,
            validation_sample,
        )

        int8_report = conversion_report(
            truth,
            fp32_scores,
            int8_scores,
            INT8_LIMITS,
            quantized=True,
        )

        format_rows.append(
            dict(
                format="INT8",
                size_MB=(
                    INT8_PATH.stat().st_size
                    / 1e6
                ),
                **int8_report,
            )
        )

        if (
            fp32_report["passed"]
            and int8_report["passed"]
        ):
            mobile_models[
                "INT8"
            ] = INT8_PATH

            if (
                INT8_PATH.stat().st_size
                < FP32_PATH.stat().st_size

```

```

        ):
            CHOSEN_FORMAT = "INT8"

    else:
        print(
            "INT8 is not selected:",
            int8_report["reason"],
        )

    except Exception as error:
        print(
            "INT8 checking failed; "
            "keeping FP32:",
            error,
        )

        (
            OUTPUT_DIR
            / "int8_validation_error.txt"
        ).write_text(
            str(error),
            encoding="utf-8",
        )

format_table = (
    pd.DataFrame(format_rows)
    .set_index("format")
)

display(format_table)

format_table.to_csv(
    OUTPUT_DIR
    / "format_validation.csv"
)

(
    OUTPUT_DIR
    / "conversion_limits.json"
).write_text(
    json.dumps(
        {
            "FP32": FP32_LIMITS,
            "INT8": INT8_LIMITS,
            "sample_images_per_class": MOBILE_IMAGES_PER_CLASS,
            "sample_method": "equal number from every expression class",
            "fp32_reference": (
                "Separate float32 Keras model on CPU "
                "with identical weights and preprocessing"
            ),
        },
        indent=2,
    ),
    encoding="utf-8",
)

if not fp32_report["passed"]:
    raise RuntimeError(
        "FP32 conversion failed the declared checks. "

        "See format_validation.csv: "
        + fp32_report["reason"]
    )

print(
    "Candidate Android format:",
    CHOSEN_FORMAT,
)

-----
NameError                                Traceback (most recent call last)
/tmp/ipykernel_76412/580567603.py in <cell line: 0>()
      1 # cell:mobile_check
----> 2 OUTPUT_DIR.mkdir(

```

```

3     parents=True,
4     exist_ok=True,
5 )

```

NameError: name 'OUTPUT_DIR' is not defined

31. Compare accuracy, size and computer speed

In this cell we compare all formats on the same holdout sample. We measure inference time on this computer's CPU. We still need separate measurements on the real Android phone.

Code cell 66

```

# cell:comparison
OUTPUT_DIR.mkdir(parents=True, exist_ok=True)
def computer_latency(path, sample):
    interpreter = tf.lite.Interpreter(model_path=str(path), num_threads=4)
    interpreter.allocate_tensors()
    details = interpreter.get_input_details()[0]
    value = sample[None].astype(np.float32)
    if np.issubdtype(details["dtype"], np.integer):
        scale, zero = details["quantization"]
        limits = np.iinfo(details["dtype"])
        value = np.clip(np rint(value / scale + zero), limits.min, limits.max)
    interpreter.set_tensor(details["index"], value.astype(details["dtype"]))
    for _ in range(10):
        interpreter.invoke()
    times = []
    for _ in range(50):
        started = time.perf_counter()
        interpreter.invoke()
        times.append((time.perf_counter() - started) * 1000)
    return float(np.median(times)), float(np.percentile(times, 95))

comparison_df = balanced_check_sample(
    test_df,
    MOBILE_IMAGES_PER_CLASS,
    SEED,
)
comparison_df.to_csv(OUTPUT_DIR / "mobile_comparison_images.csv", index=False)
comparison_labels = comparison_df["label"].to_numpy()
keras_comparison = cpu_predictions(comparison_df)
comparison_rows = [{"format": "Keras", "images": len(comparison_df),

                    "accuracy": accuracy_score(comparison_labels, keras_comparison.argmax(axis=1)),
                    "macro_f1": f1_score(comparison_labels, keras_comparison.argmax(axis=1),
                                           average="macro", labels=list(range(N_CLASSES)), zero_division=0),
                    "size_MB": (OUTPUT_DIR / "best_model.h5").stat().st_size / 1e6,
                    "computer_CPU_median_ms": np.nan, "computer_CPU_p95_ms": np.nan}]
sample, _ = next(iter(make_dataset(validation_sample.head(1))))
for name, path in mobile_models.items():
    scores = lite_predictions(path, comparison_df)
    median_ms, p95_ms = computer_latency(path, sample[0].numpy())
    comparison_rows.append({"format": name, "images": len(comparison_df),
                           "accuracy": accuracy_score(comparison_labels, scores.argmax(axis=1)),
                           "macro_f1": f1_score(comparison_labels, scores.argmax(axis=1), average="macro",
                                                  labels=list(range(N_CLASSES)), zero_division=0),
                           "size_MB": path.stat().st_size / 1e6,
                           "computer_CPU_median_ms": median_ms, "computer_CPU_p95_ms": p95_ms})
comparison = pd.DataFrame(comparison_rows)
display(comparison)
comparison.to_csv(OUTPUT_DIR / "model_comparison.csv", index=False)
print("Timing: this computer's CPU, 4 threads, 10 warm-ups, 50 runs.")
print("Image loading, cropping and resizing are not included. These are NOT phone timings.")

```

No saved output for this cell.

32. Save the Android model and its input rules

In this cell we save the chosen TensorFlow Lite model, the label order and the input rules. Android must use the same RGB face crop and image size.

Code cell 68

```
# cell:save
OUTPUT_DIR.mkdir(parents=True, exist_ok=True)
CHOSEN_PATH = OUTPUT_DIR / f"{TASK}_android.tflite"
shutil.copyfile(mobile_models[CHOSEN_FORMAT], CHOSEN_PATH)
(OUTPUT_DIR / "labels.txt").write_text("\n".join(CLASS_NAMES) + "\n", encoding="utf-8")
interpreter = tf.lite.Interpreter(model_path=str(CHOSEN_PATH))
interpreter.allocate_tensors()

def tensor_rules(details):
    return {"shape": details["shape"].tolist(), "dtype": np.dtype(details["dtype"]).name,
            "scale": float(details["quantization"][0]), "zero_point":
int(details["quantization"][1])}

settings = {
    "course": "DLBAIPEAI", "task": "Task 2", "model_task": TASK,
    "architecture": "EfficientNetV2B0", "dataset": "ExpW",
    "label_kaggle_handle": LABEL_HANDLE, "image_kaggle_handle": IMAGE_HANDLE, "seed": SEED,
"class_names": CLASS_NAMES,
    "input": tensor_rules(interpreter.get_input_details()[0]),
    "output": tensor_rules(interpreter.get_output_details()[0]),
    "image_rule": "Crop face, convert to RGB, resize to 224x224 using bilinear interpolation.",
    "pixel_rule": "Float pixels 0-255. Scaling is already inside the model. Do not divide by 255.",
    "integer_rule": "For integer tensors use the saved scale and zero point, with rounding and
clipping.",
    "includes_face_detector": False, "measured_on_phone": False,
    "split_sizes": {name: len(frame) for name, frame in frames.items()},
    "chosen_stage": BEST_STAGE, "chosen_format": CHOSEN_FORMAT,
    "training_seconds": head_seconds + fine_seconds,
    "gpu_name": GPU_NAME, "gpu_forward_backward_test_passed": GPU_READY,
    "holdout_scores": model_scores,

    "split_method": "StratifiedGroupKFold(10): fold 0 holdout, fold 1 validation, folds 2-9
training; grouped by original image path",
    "checkpoint_rule": "maximum validation balanced accuracy; validation loss breaks a tie",
    "conversion_sample_images_per_class": MOBILE_IMAGES_PER_CLASS,
    "versions": {name: metadata.version(name) for name in packages},
    "system": platform.platform(), "tensorflow_devices": [str(device) for device in gpus],
}
(OUTPUT_DIR / "model_settings.json").write_text(json.dumps(settings, indent=2), encoding="utf-8")
print("Android model:", CHOSEN_PATH)
print("The training input starts with a cropped face. The app must do this crop too.")
```

No saved output for this cell.

33. Summarise this run

In this cell we summarise the measured results. The notebook does not create results before training. The Android app and the required 20-image device experiment still need to be completed.

Code cell 70

```
# cell:summary
display(Markdown(
    f"### Measured results\n\n"
    f"The model was trained on **{len(train_df):,} images**. "
    f"The validation set had **{len(val_df):,} images**. "
    f"The final holdout had **{len(test_df):,} images**.\n\n"
    f"Holdout accuracy was **{model_scores['accuracy']:.2%**". "
    f"Macro F1 was **{model_scores['macro_f1']:.3f}**". "
    f"The selected checkpoint was **{BEST_STAGE}**", and the candidate phone format was
**{CHOSEN_FORMAT}**.\n\n"
    f"Review the class report and mistakes above. Exact duplicates were checked, "
    f"but near-duplicates and all repeated people were not ruled out. "
    f"ExpW does not provide a fixed train/test split here, so this notebook uses a fixed seeded
group-aware split. "
    f"Faces from the same original photograph cannot cross training, validation and holdout sets. "
    f"These computer tests do not replace on-phone tests."
))
```

No saved output for this cell.

What remains for Task 2

The final phone app must recognise age, gender and expression on the device. Test it with the 20 device-captured images required in the task brief. Do not use those images for training or model selection. Record consent and protect personal images. Include real phone measurements in the final report (IU International University, n.d., pp. 3-4).

The source code, critical review of the results and AI-use statement still belong in the course submission. A working model alone does not guarantee a particular grade.

Sources

IU International University. (n.d.). *DLBAIPEAI - Project Edge AI: Task - Project report* (Task 2, pp. 3-4). Course document supplied with this project.

Zhang, Z., Luo, P., Loy, C. C., & Tang, X. (2018). From facial expression recognition to interpersonal relation prediction. *International Journal of Computer Vision*, 126*(5), 550-569.
<https://doi.org/10.1007/s11263-017-1055-1>

Kaggle. (n.d.). *Expression in-the-Wild (ExpW) Dataset*. Label dataset mirror used by this notebook:
<https://www.kaggle.com/datasets/nguhaduong/expression-in-the-wild-expw-dataset>

Kaggle. (n.d.). *origin expw*. Original image mirror used by this notebook:
<https://www.kaggle.com/datasets/minhtmnguyntn/origin-expw>

Tan, M., & Le, Q. V. (2021). EfficientNetV2: Smaller models and faster training. *Proceedings of Machine Learning Research*, 139*, 10096-10106. <https://proceedings.mlr.press/v139/tan21a.html>

TensorFlow. (n.d.). *Install TensorFlow with pip*. <https://www.tensorflow.org/install/pip>

TensorFlow. (n.d.). *EfficientNetV2B0*.
https://www.tensorflow.org/api_docs/python/tf/keras/applications/EfficientNetV2B0

TensorFlow. (n.d.). *Post-training integer quantization*.
https://www.tensorflow.org/lite/performance/post_training_integer_quant

All result tables and graphs in this notebook are produced from the loaded data when the cells run.

34. Make the notebook PDF

In this cell we automatically create a PDF of the saved notes, code and recorded outputs. Save any changes to the notes first. Training is not run again. No LaTeX, browser installer or helper file is needed.

Code cell 74

```
# cell:pdf
OUTPUT_DIR.mkdir(
    parents=True,
    exist_ok=True,
)

import base64
import textwrap
import copy
import re
import io

from html import escape

import nbformat

from reportlab.lib.pagesizes import A4
from reportlab.lib.styles import (
    getSampleStyleSheet,
    ParagraphStyle,
```

```

)
from reportlab.platypus import (
    SimpleDocTemplate,
    Paragraph,
    Preformatted,
    Spacer,
    Image as PDFImage,
)

# Get the current notebook directly from Google Colab.
try:
    from google.colab import _message

    response = _message.blocking_request(
        "get_ipynb",
        timeout_sec=30,
    )

    notebook_data = response.get(
        "ipynb"
    )

    if notebook_data is None:
        raise RuntimeError(
            "Colab did not return the current notebook."
        )

    notebook = nbformat.from_dict(
        notebook_data
    )
except Exception as error:
    raise RuntimeError(
        "Could not read the current Colab notebook."
    ) from error

# Colab can return cell source as either a string
# or a list of strings. Make every source one string.
for cell in notebook.cells:

    source = cell.get(
        "source",
        "",
    )

    if isinstance(
        source,
        list,
    ):
        source = "".join(
            source
        )

    elif not isinstance(
        source,
        str,
    ):
        source = str(
            source
        )

    cell["source"] = source

    if cell.cell_type == "code":

        key = (
            source.splitlines()[0].strip()
            if source
            else ""
        )

        cell.outputs = []

```

```

cell.execution_count = None

if key in pdf_recorder.records:

    record = copy.deepcopy(
        pdf_recorder.records[key]
    )

    cell.outputs = nbformat.from_dict(
        {
            "outputs": record.get(
                "outputs",
                [],
            )
        }
    ).outputs

    cell.execution_count = record.get(
        "execution_count"
    )

# PDF styles.
styles = getSampleStyleSheet()

styles.add(
    ParagraphStyle(
        "Notes",
        fontName="Helvetica",
        fontSize=10,
        leading=14,
        spaceAfter=8,
    )
)

styles.add(
    ParagraphStyle(
        "NotebookCode",
        fontName="Courier",
        fontSize=8,
        leading=10.5,
        spaceAfter=7,
    )
)

styles["Heading1"].fontSize = 18
styles["Heading1"].leading = 22

styles["Heading2"].fontSize = 12
styles["Heading2"].leading = 16

story = []

def plain(value):
    if isinstance(
        value,
        list,
    ):
        text = "".join(
            value
        )
    else:
        text = str(
            value
        )

    # Remove terminal colour codes.
    text = re.sub(
        r"\x1b\[([0-?]*[ -/]*[@-~])",
        "",
        text,
    )

```



```

)

replacements = {
    "_": "-",
    "-": "-",
    "->": "->",
    "'": '"',
    '"': "'",
    "": ' ',
    "": ' ',
}

for old, new in replacements.items():
    text = text.replace(
        old,
        new,
    )

return (
    text
    .encode(
        "latin-1",
        errors="replace",
    )
    .decode(
        "latin-1"
    )
)

def inline(text):
    text = escape(
        plain(text)
    )

    text = re.sub(
        r"\([([^\]]+)\)\((https?://[^\s])+\)",
        r'<link href="\2">\1</link>',
        text,
    )

    text = re.sub(
        r"\*(.+?)\*\*",
        r"<b>\1</b>",
        text,
    )

    text = re.sub(
        r"`([^\`]+)`",
        r'<font name="Courier">\1</font>',
        text,
    )

    return text

def add_code(text):
    lines = []

    for line in plain(
        text
    ).splitlines():
        wrapped = textwrap.wrap(
            line.expandtabs(4),
            98,
            replace_whitespace=False,
            drop_whitespace=False,
            break_on_hyphens=False,
        )

        lines.extend(

```

```

        wrapped or [""]
    )

for start in range(
    0,
    len(lines),
    22,
):
    story.append(
        Preformatted(
            "\n".join(
                lines[
                    start:start + 22
                ]
            ),
            styles["NotebookCode"],
        )
    )

```

```

def add_notes(text):
    for paragraph in plain(
        text
    ).split(
        "\n\n"
    ):
        paragraph = paragraph.strip()

        if not paragraph:
            continue

        heading = re.match(
            r"^(#{1,3})\s+(.*)$",
            paragraph,
            re.S,
        )

        if heading:
            style = styles[
                "Heading1"
                if len(
                    heading[1]
                ) == 1
                else "Heading2"
            ]

            story.append(
                Paragraph(
                    inline(
                        heading[2]
                    ),
                    style,
                )
            )
        else:
            story.append(
                Paragraph(
                    inline(
                        paragraph
                    ).replace(
                        "\n",
                        "<br/>",
                    ),
                    styles["Notes"],
                )
            )

```

```

# Add every notebook cell to the PDF.

```

```

for number, cell in enumerate(
    notebook.cells,
    1,
):
    if cell.cell_type == "markdown":
        add_notes(
            cell.source
        )

    elif cell.cell_type == "code":
        status = (
            "recorded"
            if cell.execution_count is not None
            else "output not recorded"
        )

        story.append(
            Paragraph(
                f"Code cell {number} - {status}",
                styles["Heading3"],
            )
        )

        add_code(
            cell.source
        )

        for output in cell.outputs:
            if output.get(
                "output_type"
            ) == "stream":
                add_code(
                    output.get(
                        "text",
                        "",
                    )
                )

            else:
                data = output.get(
                    "data",
                    {},
                )

                if "image/png" in data:
                    encoded = data[
                        "image/png"
                    ]

                    if isinstance(
                        encoded,
                        list,
                    ):
                        encoded = "".join(
                            encoded
                        )

                    image = PDFImage(
                        io.BytesIO(
                            base64.b64decode(
                                encoded
                            )
                        )
                    )

                    scale = min(
                        500 / image.imageWidth,

```

```

        330 / image.imageHeight,
        1.0,
    )

    image.drawWidth *= scale
    image.drawHeight *= scale

    story.append(
        image
    )

    story.append(
        Spacer(
            1,
            8,
        )
    )

    elif "text/markdown" in data:

        add_notes(
            data[
                "text/markdown"
            ]
        )

    elif "text/plain" in data:

        add_code(
            data[
                "text/plain"
            ]
        )

# Save the PDF.
PDF_PATH = (
    OUTPUT_DIR
    / (
        NOTEBOOK_NAME
        + ".pdf"
    )
)

def page_number(
    canvas,
    document,
):

    canvas.setFont(
        "Helvetica",
        8,
    )

    canvas.drawString(
        40,
        22,
        "DLBAIPEAI - Task 2 - "
        + TASK.title()
        + " model",
    )

    canvas.drawRightString(
        A4[0] - 40,
        22,
        f"Page {document.page}",
    )

SimpleDocTemplate(
    str(PDF_PATH),
    pagesize=A4,

```

```

        leftMargin=40,
        rightMargin=40,
        topMargin=38,
        bottomMargin=38,
        title=NOTEBOOK_NAME,
        author="W. Pretorius",
    ).build(
        story,
        onFirstPage=page_number,
        onLaterPages=page_number,
    )

# Save an executed notebook copy too.
EXECUTED_PATH = (
    OUTPUT_DIR
    / (
        NOTEBOOK_NAME
        + "_executed.ipynb"
    )
)

nbformat.write(
    notebook,
    EXECUTED_PATH,
)

print(
    "Notebook PDF saved:",
    PDF_PATH,
)

print(
    "Executed notebook saved:",
    EXECUTED_PATH,
)

```

No saved output for this cell.

Download the Colab results

In this cell we put the trained models, result tables, graphs and PDF into one ZIP file and download it to the computer.

Code cell 76

```

# cell:pdf_after_recovery
# Export the saved notebook content. No training is run.

import sys
import subprocess
import importlib.util

# Install only missing PDF tools.
for package in ("nbformat", "reportlab"):
    if importlib.util.find_spec(package) is None:
        subprocess.check_call([
            sys.executable, "-m", "pip", "install", "--quiet", package
        ])

import json
import re
import io
import base64
import textwrap

from pathlib import Path
from datetime import datetime
from html import escape

import nbformat

```

```

from google.colab import _message, files as colab_files
from reportlab.lib.pagesizes import A4
from reportlab.lib.styles import getSampleStyleSheet, ParagraphStyle
from reportlab.platypus import (

    SimpleDocTemplate,
    Paragraph,
    Preformatted,
    Spacer,
    Image as PDFImage,
)

# -----
# 1. Read the current notebook without running any of its cells.
# -----
response = _message.blocking_request(
    "get_ipynb",
    timeout_sec=60,
)

if not isinstance(response, dict) or not response.get("ipynb"):
    raise RuntimeError(
        "Colab did not return the notebook. "
        "Keep this tab open and try again."
    )

payload = response["ipynb"]

notebook = nbformat.reads(
    payload if isinstance(payload, str) else json.dumps(payload),
    as_version=4,
)

# Keep the outputs already stored in the notebook.
# Do not clear them or use the old pdf_recorder.

export_dir = (
    Path("/content/project_edge_ai/notebook_exports")
    / datetime.now().strftime("%Y%m%d_%H%M%S_%f")
)
export_dir.mkdir(parents=True, exist_ok=True)

name = "03_Expression_Model_Colab"

snapshot_path = export_dir / (name + "_snapshot.ipynb")
pdf_path = export_dir / (name + ".pdf")

# Save a notebook backup before creating the PDF.
nbformat.write(notebook, str(snapshot_path))

# -----
# 2. Prepare the PDF layout.
# -----
styles = getSampleStyleSheet()

styles.add(ParagraphStyle(
    "NotebookNotes",
    fontSize=10,
    leading=14,
    spaceAfter=8,
))

styles.add(ParagraphStyle(
    "NotebookCode",
    fontName="Courier",
    fontSize=7.5,
    leading=10,
    spaceAfter=8,
))

story = []

```

```

def text(value):
    """Convert stored notebook text into printable text."""
    value = "".join(value) if isinstance(value, list) else str(value)

    # Remove terminal colour codes.
    value = re.sub(r"\x1b\[([0-9]*)[ -/]*[@-~]", "", value)
    value = value.replace("\r\n", "\n").replace("\r", "\n")

    replacements = {
        "->": "->",
        "-": "-",
        "-": "-",
        "'": "'",
        '"': '"',
        '"': '"',
        '"': '"',
        '"': '"',
        "#": "#",
    }

    for old, new in replacements.items():
        value = value.replace(old, new)

    return value.encode(
        "latin-1", errors="replace"
    ).decode("latin-1")

def add_code(value):
    """Wrap long code and output lines so they fit the page."""
    lines = []

    for line in text(value).expandtabs(4).splitlines():
        lines.extend(
            textwrap.wrap(
                line,
                width=100,
                replace_whitespace=False,
                drop_whitespace=False,
                break_on_hyphens=False,
            ) or [""]
        )

    for start in range(0, len(lines), 30):
        story.append(Preformatted(
            "\n".join(lines[start:start + 30]),
            styles["NotebookCode"],
        ))

def add_notes(value):
    """Add headings, simple notes and fenced code blocks."""

    blocks = re.split(
        r"(```.*?```)",
        text(value),
        flags=re.S,
    )

    for block in blocks:
        if block.startswith("```"):
            add_code("\n".join(block.splitlines()[1:-1]))
            continue

        for paragraph in re.split(r"\n\s*\n", block):
            paragraph = paragraph.strip()

            if not paragraph:
                continue

            heading = re.match(
                r"^(#{1,6})\s+(.+)$",
                paragraph,
                re.S,
            )

            style = styles["NotebookNotes"]

```

```

        if heading:
            paragraph = heading.group(2)
            style = styles[
                "Heading1"
                if len(heading.group(1)) == 1

                else "Heading2"
            ]

            rendered = escape(paragraph)
            rendered = re.sub(
                r"\*(.+?)\*",
                r"<b>\1</b>",
                rendered,
            )

            story.append(Paragraph(
                rendered.replace("\n", "<br/>"),
                style,
            ))

# -----
# 3. Add the saved notes, code, tables, graphs and error messages.
# -----
stored_outputs = sum(
    len(cell.get("outputs", []))
    for cell in notebook.cells
)

add_notes("# Expression model - notebook export")

add_notes(
    "This PDF contains the notebook's stored notes, code and outputs. "
    "No cells were rerun. Missing outputs are not recreated."
)

for number, cell in enumerate(notebook.cells, 1):
    if cell.cell_type == "markdown":
        add_notes(cell.source)

    elif cell.cell_type == "code":
        story.append(Paragraph(
            f"Code cell {number}",
            styles["Heading3"],
        ))

        add_code(cell.source)

        outputs = cell.get("outputs", [])

        if not outputs:
            add_notes("No saved output for this cell.")

        for output in outputs:
            kind = output.get("output_type")

            if kind == "stream":
                add_code(output.get("text", ""))

            elif kind == "error":
                traceback_text = "\n".join(
                    output.get("traceback", [])
                )

                add_code(
                    traceback_text
                    or f"{output.get('ename')}: {output.get('evalue')}}"
                )

            else:
                data = output.get("data", {})

                image_type = next(

```



```

        (
            key
            for key in ("image/png", "image/jpeg")
            if key in data
        ),
        None,
    )

    if image_type:
        image_bytes = base64.b64decode(
            text(data[image_type])
        )

        image = PDFImage(io.BytesIO(image_bytes))

        scale = min(
            495 / image.imageWidth,
            350 / image.imageHeight,
            1,
        )

        image.drawWidth = image.imageWidth * scale
        image.drawHeight = image.imageHeight * scale

        story.extend([image, Spacer(1, 8)])

    elif "text/markdown" in data:
        add_notes(data["text/markdown"])

    elif "text/plain" in data:
        add_code(data["text/plain"])

    else:
        add_notes(
            "This rich output is kept in the notebook snapshot "
            "but is not rendered in this PDF."
        )

    elif cell.cell_type == "raw":
        add_code(cell.source)

# -----
# 4. Save and download the PDF.
# -----
def page_number(canvas, document):
    canvas.setFont("Helvetica", 8)

    canvas.drawString(
        40,
        22,
        "DLBAIPEAI - Task 2 - Expression model",
    )

    canvas.drawRightString(
        A4[0] - 40,
        22,
        f"Page {document.page}",
    )

SimpleDocTemplate(
    str(pdf_path),
    pagesize=A4,
    leftMargin=40,
    rightMargin=40,
    topMargin=38,
    bottomMargin=38,
    title=name,
    author="W. Pretorius",
).build(
    story,
    onFirstPage=page_number,

```

```
        onLaterPages=page_number,  
    )  
  
    print("Stored outputs found:", stored_outputs)  
    print("Notebook snapshot:", snapshot_path)  
    print("PDF saved:", pdf_path)  
    print("No training was run.")  
  
    colab_files.download(str(pdf_path))
```

No saved output for this cell.